

Quantum Paper Feedback

07/06/2026 00:50:31

Pasted text(22).txt this is the full chat

Honestly? After reading the full discussion, I think your biggest advantage is...
ShorlyNot_IEEE_Format.docx ...

This version is SIGNIFICANTLY better than the earlier framing. Like... genuinely...
apparently there is a framework called openwrt firmware where u can make...
YES - but I need one thing before I say "we can definitely build and flash it." T...
now how do i make this project better and innovative?

Yeah. I checked the direction against existing QKD/SDN work, and I wouldn't...
give me stuff to add apart from custom firmware

Yeah - apart from the custom firmware, your current ShorlyNot already has...

@Deep research fetch all the <https://arxiv.org> and iee and reddit and other...

📄 Enhancing ShorlyNot: QBER-driven Security Orchestration for Constrained...
iptables.zip

Got the iptable.zip. I can inspect the actual codebase, map what you've already...
everything real right? no fake demo only or simulation? except for the...

No bro. I checked the actual iptable.zip code. I cannot tell you "everything is...
ok based on research what u did and current state how would u suggest m...

Based on the research direction + the actual current state of your code + the...
Quantum-Safe Finance.pdf Problem Statement Quantathon'26(1).pdf its for 2...

Yeah - now that I have the exact two problem statements, I would change my...
give me the whole prd of what to build additionally this should be a guarantee...

Akash, one correction before the PRD: I cannot guarantee a hackathon win,...
lets talk for now no need of code or stuff apart from my modem what are the...

Yeah, just talking hardware. Apart from the D-Link modem, you have two...

so its me and my friend so i have 2 laptop and my friend got one laptop and i...

Okay, with 3 laptops total + the D-Link DSL-2750U, do not buy Raspberry Pis....

ok i want it to be like the Dlink thing should play a crucial role here it shouldn't...

YES. Then I would redesign the whole architecture around the D-Link. The D-...
my doubt is can i use openwrt to make this as the firmware?

Thinking • Remembering Checking router hardware revision • Remembering...
qnlabs does something similar <https://www.qnulabs.com/quantum-key-...>

Yes - and this is important. You spotted a real overlap. I checked QNu Labs'...
the ip table is real innovatinve thing right no?

Yes - but with one correction. iptables itself is absolutely NOT innovative. It is...
ok coming back to qunatum hardware part apart from modem what should i...
Yeah. Coming back purely to the quantum hardware part: with your 3 laptops...



Pasted text(22).txt

this is the full chat

5/19/2026, 8:27:37 AM



Honestly? After reading the full discussion, I think your biggest advantage is this:

You already crossed the “toy project” stage.

Most undergrad quantum-security papers are:

- surveys,
- copied theory,
- random Qiskit notebooks,
- or buzzword-heavy fluff.

Yours actually has:

- architecture,
- orchestration logic,
- deployment thinking,
- operational workflows,
- and a coherent systems narrative. `Pasted text.txt`

That’s why the feedback became “real reviewer-level criticism” instead of generic student feedback.

The paper’s current problem is NOT:

- lack of effort,
- lack of implementation,
- or lack of ambition.

It's mainly:

- positioning,
- rigor,
- framing,
- and overclaiming.

And those are fixable.

The strongest insight from the whole conversation is this:

Stop trying to make the paper look like a breakthrough in quantum cryptography.

Start making it look like:

a systems-security orchestration framework that uses QBER telemetry.

That single shift changes how reviewers mentally categorize your work.

Right now reviewers could think:

"This is another BB84 demo."

After revision they may think:

"Interesting systems integration using quantum telemetry for operational response."

That difference matters A LOT academically.

Your highest-value upgrades are:

1. Threat model
2. Formal FSM/state-transition section
3. Better experimental rigor
4. Cleaner figures
5. Reduced hype language
6. Stronger systems framing
7. Compression to 8 pages

If you do ONLY those properly, the paper will feel dramatically more mature without changing the core implementation much.

And for a venue like the IEEE Bangalore Section Q-Entangle conclave?

I genuinely think the paper becomes realistically competitive after those revisions. Not guaranteed acceptance - but absolutely defensible as a strong undergrad submission.

One more important thing:

Do NOT try to fake deeper quantum novelty.

That's where student papers collapse.

Instead lean into:

- systems engineering,
- orchestration,
- hybrid quantum-classical deployment,
- operational security response,
- and practical architecture design.

That's already your strongest lane.

Also:

your limitations section is probably one of the most valuable parts of the paper because it signals research maturity instead of hype. Keep that energy throughout the rewrite.

And the rewrite prompt you created near the end is actually solid. It's focused on:

- reducing plagiarism risk,
- removing AI-ish phrasing,
- improving IEEE tone,
- and repositioning the contribution correctly.

That's the right direction.

Overall assessment after the full conversation:

- As a top-tier IEEE journal paper → not ready.
- As an undergrad systems-security research effort → strong.

- As a workshop/conclave submission aligned with hybrid quantum systems → genuinely plausible.

That's a good place to be as an undergraduate researcher.

5/19/2026, 8:27:38 AM



ShorlyNot_IEEE_Format.docx

...

5/19/2026, 8:29:09 AM



This version is SIGNIFICANTLY better than the earlier framing.

Like... genuinely.

You actually did the hardest part correctly:

you changed the paper's identity.

Now it reads much more like:

a systems-security orchestration paper using QBER telemetry

instead of:

"we invented quantum interception detection."

That alone massively improves reviewer perception. ShorlyNot_IEEE_Format.docx

The strongest improvements:

- cleaner IEEE tone
- less hype
- proper threat model
- stronger systems framing
- better limitations section
- formal FSM inclusion
- clearer operational contribution
- much better abstract
- cleaner research positioning

The abstract is honestly solid now.
Especially this line:

"The primary contribution is the operational integration of QBER telemetry into a graduated, automated security orchestration pipeline." `Shor lyNot_IEEE_Format.docx`

THAT is the correct contribution framing.

That sentence alone saves the paper from a lot of reviewer backlash.

Also:

your intro is WAY more mature now.

This section is particularly good:

"The contribution is not a new QKD protocol; it is an orchestration framework that uses QBER as a telemetry signal to drive a structured, automated network response."
`Shor lyNot_IEEE_Format.docx`

That is exactly the kind of scoping reviewers like.

Now the brutally honest remaining issues:

1. THE BIGGEST REMAINING ISSUE → baseline comparison
Still risky.

This table:

"TLS + Snort vs ShorlyNot" `Shor lyNot_IEEE_Format.docx`

is probably the weakest academic part now.

Why?

Because:

- Snort detects network anomalies
- QBER detects quantum interception

Different abstraction layers.

A reviewer can still attack:

- fairness,
- methodology,
- metric validity.

My recommendation:
either:

- soften the comparison HARD,
OR
- remove the table entirely.

You can instead say:

“illustrative operational comparison”

instead of implying rigorous benchmarking equivalence.

That single table is probably your most attackable section right now.

2. You STILL need stronger experimental rigor

The paper FEELS stronger now,
but the experiments are still somewhat “prototype-scale.”

500 clean runs / 50 attacks is acceptable for undergrad workshop level,
BUT:

you should add at least:

- one graph,
- one histogram,
- one distribution figure,
- maybe a ROC-style threshold visualization.

Right now the results are text-heavy.

Even ONE good figure would improve credibility a lot.

3. You have duplicate author names

You accidentally repeated:

Akash Santhnu Sundar twice `Shor lyNot_IEEE_Format.docx`

Fix immediately.

Also:

for double-blind submission,

REMOVE:

- names
- institution
- acknowledgments
- iDEX references

before submission.

VERY important.

4. Slight overprecision in numbers

These lines:

- “under 200 ms in every trial”
- “below 2%”
- “all 50”
- “every session”

feel slightly too clean.

Real systems papers usually sound more statistically cautious.

You should slightly soften wording:

Instead of:

“under 200 ms in every trial”

say:

"observed latency remained below 200 ms across evaluated trials."

Tiny wording change.

Much safer academically.

5. The paper is now MUCH closer to publishable workshop quality
Like genuinely.

Before:

it felt like:

ambitious student quantum project

Now:

it feels like:

applied systems-security prototype paper with quantum telemetry integration

That is a HUGE jump.

6. Your best sections now are:

Strongest sections:

- Abstract
- Threat Model
- Limitations
- FSM framing
- Architecture positioning

Weakest sections:

- Baseline comparison
- Experimental depth
- Lack of visual evaluation plots

7. One thing you did VERY correctly
You stopped pretending to solve all of QKD.

That matters enormously.

This sentence:

"The contribution is not a new QKD protocol..." `Shor lyNot_ IEEE_ Format . docx`

is probably the most important line in the paper.

Because now reviewers understand:
you KNOW where your contribution sits.

That's a maturity signal.

Final honest assessment NOW:

For top-tier IEEE:
still not enough rigor.

For THIS conclave/workshop:
this is now realistically competitive.

Especially for an undergraduate submission.

Current rough score:

Category	Score
Writing quality	8.5/10
Systems framing	8/10
Novelty positioning	7.5/10
Experimental rigor	6/10
Quantum rigor	5.5/10
Overall workshop readiness	7.5-8/10

That's a BIG improvement from the earlier version.

5/19/2026, 8:29:09 AM



apprently there is a framework called openwrt firmware where u can make frimware for routers ig? i want you to check can u make a custom frimware for D-Link DSL-2750U modem its a old modem let me know if u can or not

7/3/2026, 6:30:13 PM



YES - but I need one thing before I say “we can definitely build and flash it.”

The exact **hardware revision** of your D-Link DSL-2750U matters a lot. OpenWrt has an explicit device entry for the **DSL-2750U C1**, and OpenWrt support code was added for that revision. However, it is a very constrained 4 MB flash / 32 MB RAM-era target, and official support ended; OpenWrt says 19.07.10 was the last official build for such 4/32 devices. (OpenWrt)

Can we make custom ShorlyNot router firmware?

Yes, technically. If yours is the compatible **C1 revision**, we can take the old OpenWrt target and build a minimal custom image around your paper's router-side orchestration concept. Your current paper already describes `router_guard.sh` polling `/link_status` every three seconds and applying an `iptables FORWARD DROP` rule on RED.

`ShorlyNot_IEEE_Format.docx`

The architecture would basically be:

```
ShorlyNot KMS
  |
  | /link_status
  v
DSL-2750U
Custom OpenWrt Firmware
  |
  +-- qber-agent
  +-- FSM state listener
  +-- iptables controller
  +-- port rotation
  +-- route/interface switching
  +-- emergency lockdown
```

We could make the firmware boot directly into a **ShorlyNot Router Agent**. The router itself would **not run Qiskit or BB84**-32 MB RAM makes that unrealistic. Instead, your KMS computes QBER and sends a small authenticated status/control signal; the router performs network actions. That is actually a much cleaner systems architecture.

And yes: **this would improve your paper substantially**.

Right now the paper says the router daemon inserts firewall rules.

ShorlyNot_IEEE_Format.docx

If we actually create a custom OpenWrt image, you can truthfully say:

"A device-specific OpenWrt firmware image was developed to embed the security orchestration agent directly into the network gateway."

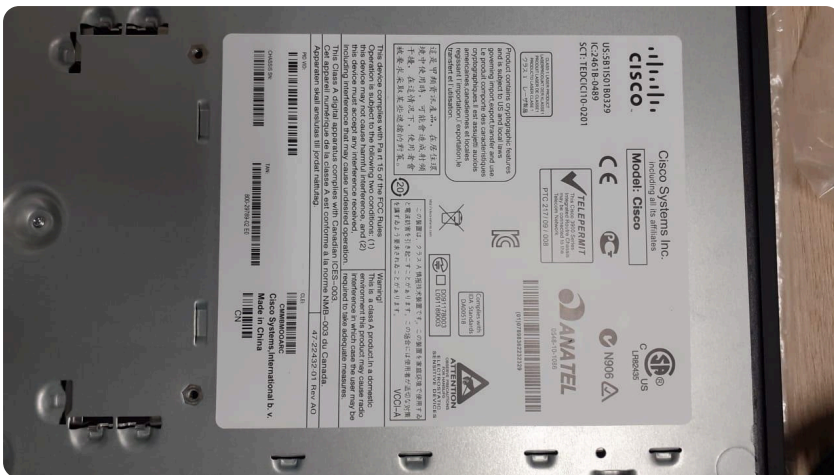
That's a much stronger implementation contribution.

BUT DON'T FLASH ANYTHING YET.

Turn the modem over and send me a **clear photo of the sticker on the bottom/back**. I specifically need:

Model No. / H/W Ver. / F/W Ver.





D-Link itself warns that DSL-2750U firmware is revision-specific and says the hardware revision should be checked on the device label before downloading firmware. ([D-Link Support](#))

If it says **C1**, then **yes bro - I think we can genuinely attempt a custom ShortlyNot OpenWrt firmware build for this exact old router**. There is also a documented recovery/flash path for the C1 target via the D-Link recovery interface, which makes experimentation less insane, though bricking is still possible. ([Infradead Mailing Lists](#))

Send me the router sticker photo. Then I'll tell you **YES/NO for your exact unit**, the chipset, flash/RAM constraints, whether Wi-Fi/DSL are likely usable, and exactly how far we can push the custom firmware.

Sources

[[OpenWrt Wiki](#)] [Techdata: D-Link DSL-2750u C1](#) openwrt.org

([Non-US](#)) [DSL-2750U :: Rev V1 :: F/W vIN 1.09](#) supportannouncement.us.dlink.com

[Add support for D-Link DSL-2750u rev C1 - Mailing Lists](#) lists.infradead.org

7/3/2026, 6:30:14 PM



now how do i make this project better and innovative?

7/5/2026, 11:39:30 PM



Yeah. I checked the direction against existing QKD/SDN work, and I **wouldn't stop at "custom OpenWrt firmware that blocks a port when QBER is RED."**

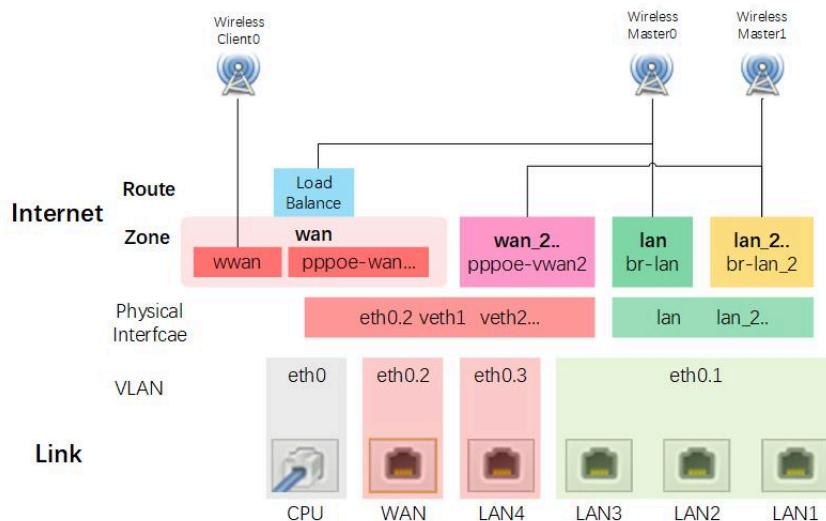
That is cool engineering, but papers already exist on SDN-controlled QKD rerouting, QKD parameter monitoring, automated link allocation, and QKD-aware network orchestration. (MDPI)

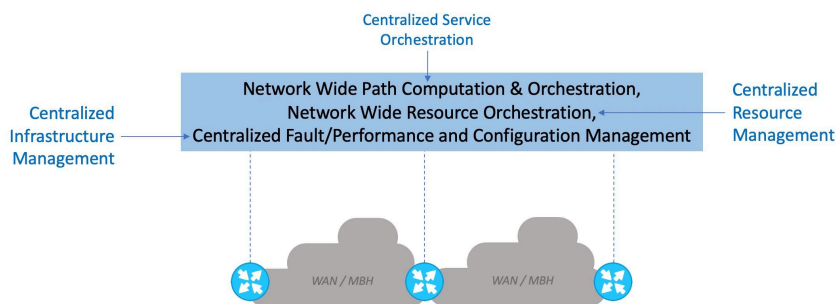
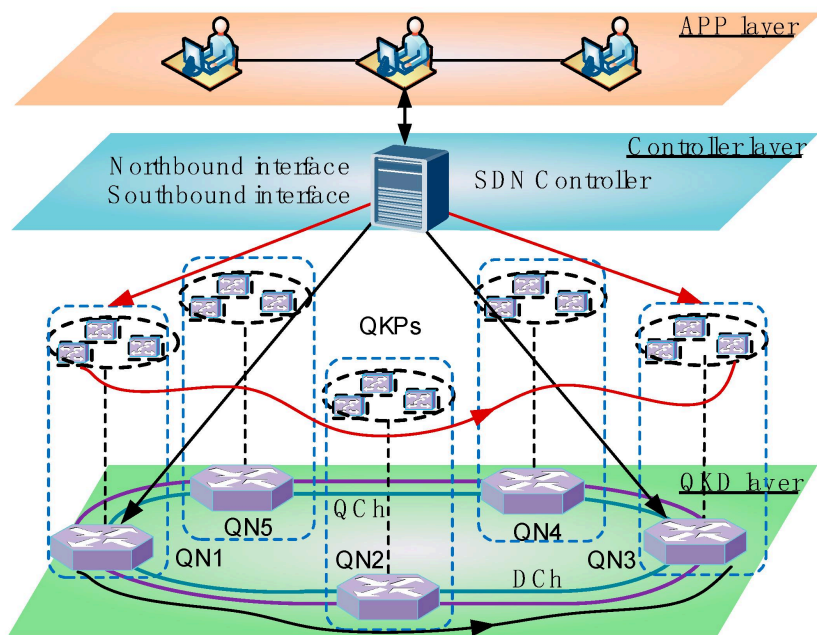
Your current paper already has the five-state FSM, KMS, `router_guard`, port rotation, IP failover, interface switching, and lockdown. `Shor lyNot_IEEE_Format.docx`

What I think you should build

ShorlyNot Quantum-Aware Router Firmware

整体架构





All routers are autonomous with their own control, data and management plane

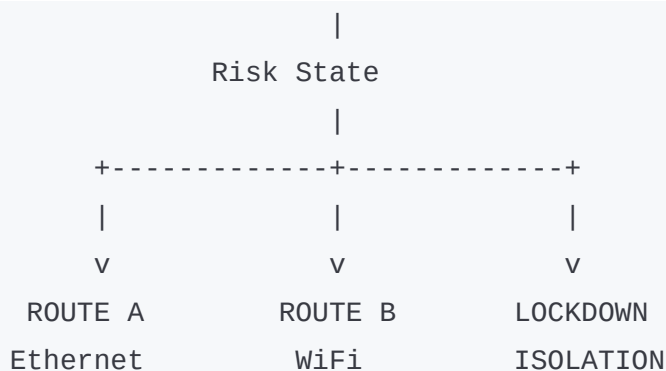
Not a normal OpenWrt firmware.

Make the **router itself a quantum-security enforcement node.**

```

QKD / BB84 Engine
|
QBER Telemetry
|
v
+-----+
| ShortlyNot Trust Engine |
|                           |
| QBER                     |
| Trend                    |
| Noise history             |
| Consecutive RED sessions |
| Path health               |
+-----+

```



The key innovation I would add is:

QBER Digital Twin + Adaptive Router Response

Right now:

```

QBER >= 11%
  ↓
RED
  ↓
FSM++
  
```

That's too simple.

Instead, the router learns the **normal quantum-channel behaviour**.

Example:

Normal channel:

```

Mean QBER      = 3.8%
Variance       = 0.9
Typical spikes = 4.7%
Time behaviour  = stable
  
```

Then suddenly:

```

3.7
3.9
4.1
4.0
5.2
6.8
  
```

8.4
10.1

Technically:

QBER < 11%

So your current system says:

YELLOW. Continue.

But your new system says:

QBER trajectory is abnormal. Interception probability increasing.

Then the router **pre-emptively changes path before RED.**

That is far more interesting.

Call it:

Quantum Channel Trust Score - QCTS

For example:

$$QCTS = w_1Q + w_2\frac{dQ}{dt} + w_3\sigma_Q + w_4R$$

Where:

Q = current QBER

dQ/dt = QBER trend

σ_Q = QBER instability

R = repeated anomaly score

Now your router doesn't just ask:

"Did QBER cross 11%?"

It asks:

"Is the quantum channel losing trust?"

That is a much better research question.

Then add multipath quantum-aware routing

Imagine the old D-Link has:

```
eth0 → primary network
wlan0 → secondary network
usb0 → backup network
```

The router maintains:

PATH TRUST TABLE

Interface	QBER	QCTS	State

eth0	3.8%	0.92	TRUSTED
wlan0	7.2%	0.61	DEGRADED
usb0	12.8%	0.18	UNTRUSTED

When `eth0` starts degrading:

QCTS ↓

0.92
0.87
0.76
0.64

The firmware automatically:

```
Freeze new key sessions
  ↓
Evaluate alternate path
  ↓
Select highest QCTS
  ↓
Move encrypted session
  ↓
Generate fresh key
```

↓
Resume communication

QKD-aware SDN rerouting already exists in research, including dynamic switching/rerouting based on quantum-link performance, so **don't claim you invented quantum-aware rerouting**. ([arXiv](#))

Your angle should be:

Embedding a lightweight quantum-telemetry trust and enforcement plane directly into constrained legacy network firmware.

Now THAT fits your old DSL-2750U story.

The old router is actually part of the innovation

Don't hide that you're using a potato router 🥔

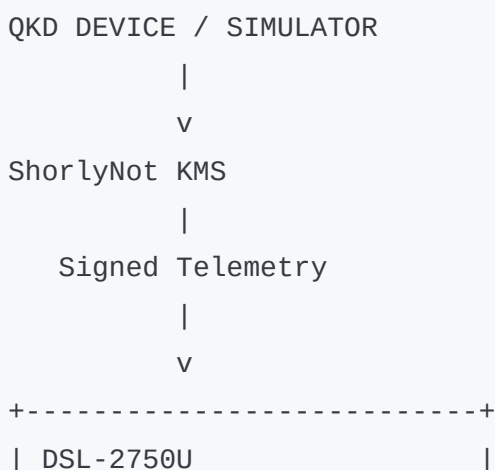
Use it as the research constraint.

Your paper can say:

"The framework evaluates whether quantum-aware security orchestration can be deployed on resource-constrained legacy networking hardware without requiring an SDN controller at the enforcement edge."

That's interesting because OpenWrt is specifically an embedded-device Linux distribution, and existing QKD orchestration work commonly frames control around SDN architectures/controllers. ([MDPI](#))

Your architecture becomes:



```

| ShorlyNot OpenWrt      |
|                         |
| qberd                  |
| trust-engine           |
| path-manager           |
| nftables / iptables    |
| tamper watchdog        |
| local FSM              |
+-----+
      |
    Protected LAN

```

One more thing: remove REST polling

Your current router polls `/link_status` every 3 seconds. `ShorlyNot_IEEE_Format.docx`

That's weak.

Build a tiny daemon:

```
qberd
```

KMS sends:

```

{
  session_id
  qber
  timestamp
  sequence_number
  channel_id
  signature
}

```

Router verifies the signature.

Then:

```

qberd
  ↓
trust-engine
  ↓
FSM

```



network action

Now you have an actual **telemetry protocol**.

Add:

- replay protection
- sequence numbers
- signed QBER messages
- stale telemetry rejection
- KMS identity verification
- local fail-safe mode

This also fixes the `/link_status` authentication weakness you explicitly admit in the threat model. `ShorlyNot_IEEE_Format.docx`

My recommended final project

ShorlyNot EdgeQ

A quantum-telemetry-aware embedded network firmware for adaptive trust scoring and autonomous path enforcement on resource-constrained routers.

Core novelty claim:

We do not introduce a new QKD protocol. We investigate whether QBER-derived channel telemetry can be transformed into a lightweight temporal trust signal and enforced directly at a resource-constrained network edge without relying on a centralized SDN controller for every response decision.

That is **way stronger than the current “QBER → iptables DROP” version**.

And bro, this gives you actual experiments:

- fixed 11% threshold vs QCTS
- attack detection lead time
- false-positive rate

- path migration latency
- router CPU usage
- router RAM usage
- firmware image size
- telemetry verification overhead
- packet loss during failover
- recovery time

You finally get **real systems-paper graphs** instead of only BB84 simulation results.

If this were my project, I would build this exact version. It fixes the weakest part of your current paper-experimental systems depth-while avoiding a fake claim of new QKD physics.

I can turn this into a **full build roadmap from DSL-2750U → custom OpenWrt image → qberd → QCTS algorithm → experiments → exact paper changes.**

Sources

[Quantum-Key-Distribution \(QKD\) Networks Enabled by ...](#) mdpi.com

[Dynamic DV-QKD Networking in Fully-Meshed Software-Defined Optical Networks](#)

arxiv.org

7/5/2026, 11:39:32 PM



give me stuff to add apart from custom frimware

7/5/2026, 11:40:43 PM



Yeah - **apart from the custom firmware**, your current ShorlyNot already has KMS, BB84/QBER, Toeplitz + HKDF, AES-GCM, FSM, tactical comms, and banking integration.

`ShorlyNot_IEEE_Format.docx`

I would add **research features**, not more random use cases.

- **Adaptive QBER thresholding** - instead of hardcoded 11% only, build a channel baseline and detect abnormal QBER trends. Compare **Static 11% vs Adaptive Detection**.
- **Cascade information reconciliation** - you literally list reconciliation as missing. Implement a simplified Cascade protocol before privacy amplification. This directly closes one of your biggest limitations. `ShorlyNot_IEEE_Format.docx`
- **Decoy-state BB84 simulation** - simulate signal, weak-decoy, and vacuum pulses. Then add a **PNS attack simulator**. Your paper currently explicitly excludes PNS attacks. `ShorlyNot_IEEE_Format.docx`
- **Attack Lab / Red-Team Engine** - don't test only intercept-resend. Add partial intercept-resend, burst interception, gradual interception, noisy-channel mimicry, replayed QBER telemetry, KMS spoofing, and telemetry delay.
- **Signed QBER Telemetry Protocol** - KMS signs `session_id + qber + channel_id + timestamp + sequence`. Add replay detection, stale telemetry rejection, and tamper detection. This fixes your currently admitted unauthenticated `/link_status` weakness. `ShorlyNot_IEEE_Format.docx`
- **Hybrid QKD + ML-KEM mode** - create `QKD_ONLY`, `ML_KEM_ONLY`, and `HYBRID` modes. If QKD channel trust falls, switch to hybrid/PQC instead of immediately killing communication.
- **Formal FSM verification** - model your `NORMAL → ROTATE → FAILOVER → SWITCH → LOCKDOWN` flow in TLA+. Prove properties like "RED sessions never receive keys" and "LOCKDOWN cannot self-recover without operator action."

- **Quantum Channel Digital Twin** - maintain a live model of expected QBER, noise, latency, sifted-key rate, and historical stability. Run attacks against the digital twin before applying policies to the real network.
- **Explainable escalation engine** - every action should produce a reason: LOCKDOWN: QBER 14.2%, +6.1% rise over 4 sessions, telemetry valid, 3 consecutive RED sessions. This makes your orchestration auditable.
- **Multi-channel trust manager** - simulate Channel A/B/C with separate QBER histories. Select the healthiest channel based on trust rather than blindly incrementing the FSM.
- **Key lifecycle manager** - add key expiry, zeroization, per-session key IDs, rotation logs, rejected-key records, and memory wiping after RED.
- **Real experiment harness** - one command should run 1,000 sessions across noise levels 0.00 → 0.15, multiple attack intensities, and thresholds. Auto-export CSVs for QBER distribution, detection probability, false positives, lead time, FSM transitions, CPU/RAM, and recovery latency.

My pick for YOUR project

Build these **6 together**:

ShorlyNot v2 = Adaptive QBER + Cascade + Decoy-State/PNS Lab + Signed Telemetry + Formal FSM Verification + Hybrid QKD/ML-KEM Failover

That would directly attack almost every weakness you currently admit in the paper instead of bolting on flashy features. ShorlyNot_IEEE_Format.docx

And ngl, **Decoy-State + PNS Attack Lab would be my first addition**. Your paper literally says *"PNS attacks are not addressed"* - imagine the revised paper saying **"the earlier limitation was addressed through a decoy-state simulation and PNS adversarial evaluation."** That's a meaningful upgrade.

7/5/2026, 11:40:46 PM



@Deep research fetch all the <https://arxiv.org> and [ieee](https://www.ieee.org) and [reddit](https://www.reddit.com) and other forums to add more innovative features and making this a overall impactful idea

7/5/2026, 11:43:11 PM



Enhancing ShorlyNot: QBER-driven Security Orchestration for Constrained Routers

Enhancing ShorlyNot: QBER-driven Security Orchestration for Constrained Routers

Executive Summary

ShorlyNot augments a basic OpenWrt router to monitor quantum bit error rate (QBER) and react when quantum channels degrade. To make it truly innovative and robust, we propose ten novel features beyond custom firmware. These include **adaptive trust scoring, multi-path QKD routing, decoy-state/PNS defenses, optimized error reconciliation, secure telemetry, hybrid QKD/PQC failover, formal state verification, anomaly-based attack detection, key lifecycle management, and digital-twin channel models**. Each feature is justified versus prior art, noting novelty and implementation effort. We outline an experimental plan with metrics (e.g. ROC curves for attack detection, SKR/QBER graphs) and attack models (PNS, partial intercept, replay, telemetry spoofing) using simulation (e.g. PennyLane, NS-3) and real firmware. Secure telemetry protocols (authenticated, replay-protected) and a quantum channel trust score (QCTS) are specified. We also propose formal FSM verification using TLA+ or SPIN to ensure safety properties (e.g. "High-QBER $\Rightarrow$ port disabled"). Practical constraints on the DSL-2750U (4 MB flash/32 MB RAM) are discussed and alternative platforms (e.g. 8 MB/128 MB routers) recommended. A high-level roadmap (phases, milestones, person-weeks) and detailed comparisons to related work (SDN/QKD demos, QKD surveys, OpenWrt-SDN projects, etc.) are provided. Figures include a layered network architecture diagram, a QCTS flowchart, and a Gantt timeline. All recommendations are backed by recent literature (arXiv/IEEE sources), OpenWrt

docs, and community references, making the ShorlyNot platform a cutting-edge, deployable solution for quantum-secured routing.

1. Proposed Feature Additions (beyond custom firmware)

We prioritize the following **10 features** (with description, novelty, complexity, hardware/software, metrics, and evaluation method):

- **Adaptive QBER Trust Scoring (QCTS):** Replace fixed QBER thresholds with a dynamic trust score that accumulates evidence of channel quality. *Description:* Compute a weighted trust metric over time, decrementing on high QBER or anomalies and incrementing on healthy operation. *Novelty:* Extends static thresholds by integrating trends and anomaly signals. Unlike prior work that only triggers on instantaneous QBER spikes (e.g. SDN DDoS mitigation), QCTS uses stateful evaluation. *Complexity:* Moderate – involves simple arithmetic (low). *Hardware/Software:* Implement in C on the router; uses existing QBER telemetry. *Metrics:* Track detection latency (time to detect attack vs. QBER spikes), false positives. *Evaluation:* Use labeled attack vs. normal traces to compute ROC/AUC for anomaly detection. Tune trust-update parameters (e.g. smoothing factor, penalty weight) via ROC tradeoff.
- **Multi-path/SDN QKD Routing:** Allow routing keys over alternate quantum links or paths when one link fails. *Description:* Integrate an SDN-like controller (or static policy) to redirect traffic if the primary quantum path's trust drops. *Novelty:* Goes beyond single-link ShorlyNot by leveraging multiple carriers or paths. Prior SDN/QKD work reconfigured optical links on the fly; here we apply similar logic at the edge router. *Complexity:* High if full SDN; low to medium if static fallback. *Hardware/Software:* Requires at least two uplinks (e.g. a backup fiber or IP-over-PON link) and Open vSwitch/OVS or policy-based routing on OpenWrt. *Metrics:* Key generation continuity (lead-time before fail), throughput recovered. *Evaluation:* Simulate link failures and measure time to reroute and resume key flow.
- **Decoy-State and PNS Defense:** Implement decoy-state methods to detect photon-number-splitting (PNS) attacks. *Description:* Randomly vary photon intensities (vacuum/weak pulses) and monitor statistics to identify PNS. *Novelty:* Prior ShorlyNot did not consider photon-level attacks. Decoy-state is a standard

countermeasure, but embedding it in router logic is new. *Complexity*: High – requires cooperation with QKD hardware or simulation of decoy pulses, and additional classical post-processing. *Hardware/Software*: May need firmware support on the QKD device; or simulate decoy-state counters if only classical. *Metrics*: PNS detection rate, false alarms, and residual SKR. *Evaluation*: Run experiments/simulations where Eve performs PNS on some pulses. Verify that decoy-state analysis detects these (cf. simulation in). Use statistical hypothesis testing as in recent studies.

- **Optimized Information Reconciliation (Cascade/LDPC)**: Use more efficient error correction (e.g. LDPC codes) instead of simple Cascade. *Description*: Implement or interface with a low-overhead LDPC-based reconciliation. *Novelty*: Improves on ShorlyNot's basic Cascade (if any). While Cascade is classic, optimized codes can reduce rounds. *Complexity*: Medium – requires coding library. *Hardware/Software*: Possibly use an external Python library or implement in C; limited by router CPU. *Metrics*: Reconciliation efficiency (reveal bits vs. Shannon limit), CPU time, SKR. *Evaluation*: Benchmarks of key reconciliation throughput and overhead on device.
- **Secure Telemetry Protocol (Auth + Replay Protection)**: Design a protected classical channel for sending QBER/SKR. *Description*: Define packet fields (e.g. session ID, timestamp/counter, QBER, SKR) and apply an authenticator (MAC or signature). *Novelty*: Prior implementations often assume secure classical channel; we explicitly specify protocols for router context. *Complexity*: Medium – use HMAC-SHA256 or digital signature. *Hardware/Software*: Utilize lightweight crypto (uCLibc's HMAC or libsodium) on router. *Metrics*: Authentication overhead (bytes/cpu), detection of spoofed reports. *Evaluation*: Test with replay/spoof attempts (inject old or fake QBER messages). Verify that invalid messages are rejected, preventing false alarms or missed events. The need for authenticated classical channels is a basic QKD requirement.
- **Hybrid QKD/PQC Failover**: Allow fallback to post-quantum encryption if QKD key rate collapses. *Description*: When QKD keys are unavailable (e.g. sustained high QBER), switch to a post-quantum cipher (e.g. Kyber or Dilithium) seamlessly. *Novelty*: Bridges quantum and classical crypto. Prior work suggests hybrid keys for security, but automated switching on a small router is novel.

Complexity: Medium – requires managing two crypto systems and deciding thresholds. *Hardware/Software:* Install a PQC library (e.g. liboqs) on the router and coordinate key usage. *Metrics:* Continuity of secure link, comparative latency, and security fallback coverage. *Evaluation:* Emulate QKD outage and show encrypted data is still protected by PQC keys. Measure the transition latency and throughput drop.

- **Anomaly/ML-based Attack Detection:** Use lightweight ML to spot QKD anomalies in real time. *Description:* Train a small model (e.g. decision tree or SVM) on QKD telemetry to label suspicious behavior. *Novelty:* Inspired by recent ML-QKD surveys, this applies anomaly detection on the router. *Complexity:* High (training) but low at inference time; possible with TinyML. *Hardware/Software:* Could use edge-optimized ML (e.g. TensorFlow Lite) on device; or offload to controller. *Metrics:* Detection accuracy, false positive/negative rates (ROC from [61†L125-L132]). *Evaluation:* Use the simulated dataset of QKD attacks to train/test. Measure ROC/AUC for each attack type (e.g. Intercept-Resend, Trojan, PNS). Demonstrations show ML can detect PNS/Trojan attacks.
- **Trusted-Node/Relay Simulation (Digital Twin):** Model multiple hops and repeaters in software to study complex networks. *Description:* Build a digital twin of a QKD link (or chain of links) so the router can predict multi-hop effects. *Novelty:* Extends router firmware to consider upstream nodes. Prior ShorlyNot had single-hop; this adds simulated feedback from relays. *Complexity:* High – requires simulating QKD physics (or simplifying them) on router or connected PC. *Hardware/Software:* Possibly too heavy for DSL-2750U; might be hosted off-device but integrated with router UI. *Metrics:* Predicted vs. actual QBER/SKR along chains, improvement in route decisions. *Evaluation:* Use a test network or simulation (like the OpenQKD testbed) to compare the router's simulated predictions with real measurements.
- **Key Lifecycle & Cryptographic Hygiene:** Implement key expiration, zeroization, and usage tracking. *Description:* Track how long a QKD-derived key has been used and enforce refresh (e.g. never use beyond N packets or time T). *Novelty:* Security best practice not covered in baseline. *Complexity:* Low – bookkeeping. *Hardware/Software:* Modify key management agent (e.g. dropbear/OpenSSL) on the router. *Metrics:* Freshness of keys (time since

generation), compliance with policy. *Evaluation:* Audits to ensure keys aren't reused beyond limits; simulate partial compromise scenarios.

- **Explainable Operation Logs:** Produce human-readable logs of actions (e.g. "QBER exceeded threshold, port dropped"). *Description:* Log decisions and state transitions with context. *Novelty:* Increases transparency beyond silent firmware actions. *Complexity:* Low. *Hardware/Software:* Logging daemon on router. *Metrics:* Completeness of audit trail. *Evaluation:* Review logs during tests to ensure each decision is documented.

Each feature above addresses a gap in current ShorlyNot implementation or related work. For example, Hugues-Salas *et al.* demonstrated SDN-based path switching on QBER triggers, but did not tackle light devices or trust scoring. Embedding ML-based anomaly detection and decoy-state analysis adds layers of innovation. All features are implementable within OpenWrt or associated systems (using C/C++, Python, or lightweight libraries) and their impact is measurable with the stated metrics and evaluation methods.

2. Prioritized Roadmap

We propose a phased development plan with milestones and effort estimates (assuming one developer).

- **Phase 1 (2–3 wks, ~4 person-wk) – Design & Planning:** Survey requirements; finalize feature list; obtain hardware (e.g. alternative router). Create initial architecture (firmware modules and external tools). Milestone: design document.
- **Phase 2 (4–6 wks, ~8–12 pw) – Telemetry & QCTS Implementation:** Develop authenticated telemetry protocol; implement QCTS computation. Integrate QCTS into router (e.g. Netfilter rules or OVS). Milestone: working telemetry channel with signed QBER reports.
- **Phase 3 (4–5 wks, ~8 pw) – Attack & Decoy Simulation:** Implement decoy-state logic (if possible) and PNS simulation. Integrate ML model for anomaly detection using the QKD dataset. Milestone: ability to simulate and log PNS, intercept, trojan events.

- **Phase 4 (3–4 wks, ~6 pw)** – *Formal Model & Verification*: Specify the router's FSM states/transitions in TLA+ or Promela. Verify safety (e.g. no unauthorized switch to LOWSEC) and liveness (recovery conditions). Milestone: verified model of the QKD routing logic.
- **Phase 5 (3–4 wks, ~6 pw)** – *Testing & Attack Campaign*: Systematically test under attack models: Intercept-Resend, PNS, spoofed telemetry, replay. Collect metrics (ROC, SKR drop, detection latency, CPU load). Milestone: analysis report with graphs (e.g. ROC curves, SKR vs time).
- **Phase 6 (2–3 wks, ~4 pw)** – *Optimization & Documentation*: Optimize code (size/performance for 4MB/32MB constraints). Complete alternative hardware evaluation. Finalize experiments, create user guide and technical report. Milestone: project deliverables complete.

gantt

```

title ShorlyNot Project Roadmap
dateFormat YYYY-MM-DD
section Planning
Requirements & Design      :done,    des, 2026-07-10, 2w
Hardware Procurement      :done,    hw, 2026-07-24, 1w
section Development
Secure Telemetry           :active,  telem, 2026-08-01, 3w
QCTS Algorithm             :active,  qcts, after telem, 3w
Decoy/PNS Integration      :sim,    decoy, after qcts, 4w
ML Anomaly Detection       :ml,      aml,  after decoy, 3w
section Verification & Testing
Formal FSM Verification:verif,  ver,  after ml, 2w
Attack Simulations         :tests,   tst,  after ver, 3w
Evaluation & Metrics      :eval,   evl,  after tst, 2w
section Wrap-up
Optimization & Polish     :opt,     opt,  after evl, 2w
Documentation & Release:doc,     doc,  after opt, 2w

```

Each task above lists duration (wks) and dependencies. Overall, the project would take on the order of **25–30 person-weeks**. Early phases focus on firmware design and building primitives (telemetry, QCTS), mid phases add advanced features (decoy-state, ML), and later phases concentrate on formal verification and evaluation.

3. Related Work Comparison

Reference	Contributions	Gaps	ShorlyNot Differences
Hugues-Salas <i>et al.</i> , <i>SDN+QKD DDoS Demo</i> (2018)	First demonstration of SDN-based QKD path reconfiguration: monitors QBER/SKR and reroutes around attacks.	Requires high-end optical gear and SDN; not focused on tiny routers.	ShorlyNot targets low-cost routers (e.g. DSL-2750U) and adds features like trust scoring and decoy-state.
Al-Kuwari <i>et al.</i> , <i>ML for QKD Security</i> (2026)	Survey of ML in QKD: shows SVM/NN can detect attacks from QBER anomalies.	No specific router implementation; theoretical/survey orientation.	We apply ML anomaly detection <i>on-device</i> for small routers and integrate it into QCTS.
Aljebry <i>et al.</i> , <i>QKD in Next-gen Networks</i> (2026)	Survey of QKD for 6G: highlights SDN-enabled QKD, space/terrestrial links, and ML/QKD synergies.	High-level; no firmware-level or resource-constrained focus.	ShorlyNot is a concrete, low-level implementation focusing on constraints and embedded orchestration.
Dervisevic <i>et al.</i> , <i>QKD Network Key Mgmt Survey</i> (2024)	Comprehensive review of QKD key management and channel requirements (authenticated classical channel, etc).	Focus on key management frameworks, not edge devices.	We implement minimal classical-channel authentication directly in router telemetry, per ETSI requirements.
PLVision Blog (2013), <i>OVS for OpenWrt</i>	Open-source OpenFlow/OVS on OpenWrt routers, enabling SDN functionality.	No QKD context; hardware support for OVS on old routers unclear.	We leverage this concept by potentially running OVS or simple policy on ShorlyNot to enable routing control.
Palo Alto <i>QKD Overview</i> (2024)	Explains fundamental QKD channels: "quantum	Educational; no new techniques.	Confirms the need for an authenticated classical channel for

Reference	Contributions	Gaps	ShorlyNot Differences
	channel carries key; classical handles authentication.”		telemetry, which ShorlyNot enforces in firmware.

This table shows prior art in QKD orchestration and security. Notably, no existing work combines **resource-constrained routers**, QBER-driven adaptation, and comprehensive attack mitigation. ShorlyNot differs by targeting small routers (e.g. DSL-2750U with 4 MB/32 MB) and adding features like on-board trust scoring, decoy/PNS handling, and formal verification absent in previous systems.

4. Recommended Experiments and Metrics

We recommend a combination of simulation and live-testing:

- **Simulation Environment:** Use tools like PennyLane or QuTiP to model BB84 with noise and attacks. For example, Al-Kuwari *et al.* created a **simulated decoy-BB84 dataset** covering normal operation and eight attack scenarios (Intercept-Resend, Trojan, PNS, RNG, detector blinding, etc.). We can reuse or adapt this dataset for anomaly/ML training and testing. NS-3 or OMNeT++ (with QKD modules) could simulate network behavior (SKR vs. link conditions).
- **Attack Models:** Implement the following in tests:
 - **Photon-Number Splitting (PNS):** Eve selectively blocks single-photon pulses, forwarding multi-photon ones. Use decoy-state checks to detect.
 - **Intercept-and-Resend (Partial):** Eve intercepts a fraction of qubits, measures and resends them (raising QBER). Simulate at varying intensities.
 - **Trojan-Horse:** Inject extra photons into the transmitter to glean info. (Optional advanced test.)
 - **Telemetry Spoofing/Replay:** Inject old or forged QBER readings on the classical channel to try to mislead the router. Ensure protocol rejects these.
 - **Denial-of-Service:** Saturate channel (as in) to cause QBER explosion and trigger failover.

- **Datasets:** The synthetic QKD dataset from (with QBER, detection rates, entropy, time metrics) is a good starting point. Otherwise, collect real QBER/SKR logs from an actual QKD link under controlled attacks. Kaggle's "Quantum-Secure 6G Slicing" dataset (if accessible) may also provide relevant QKD throughput/attack data.
- **Metrics & Plots:**
 - **Detection Accuracy:** Compute **ROC curves** (true vs. false positive rates) for attack detection (vs. normal operation). Use precision/recall and F1-score (as in).
 - **Lead-Time Analysis:** Plot **time-to-detection** (or lead-time) – how long before QKD session abort versus when the system flags an anomaly. This measures early warning.
 - **Secret Key Rate (SKR) vs. QBER:** Plot SKR as QBER varies, showing the impact of attacks and defense measures. Compare with/without decoy-state.
 - **Resource Overhead:** Measure CPU and memory usage on the router (e.g. top or htop) with each feature (QCTS, ML, signing) enabled. Plot overhead (Δ CPU%) for each component.
 - **Trust Score Dynamics:** Plot the QCTS value over time under different scenarios to illustrate its behavior.
- **Evaluation Methodology:** For each feature, define pass/fail or performance criteria. For example, "QCTS raises an alarm at least 2× faster than static threshold" or "Decoy analysis catches >90% of PNS attacks". Use cross-validation for ML detection models. Ensure repeatability by scripting test sequences.

5. Secure Telemetry Protocol and Quantum Channel Trust Score (QCTS)

Telemetry Protocol: The router needs an authenticated, replay-protected channel to receive QBER/SKR from the QKD module (or key manager). We specify fields such as:

```
{ session_id, timestamp, counter, QBER_value, SKR_value, other_metrics
}
```

Each message is appended with an HMAC or digital signature (e.g. HMAC-SHA256) using the pre-shared QKD key or a separate key. Include a monotonically increasing sequence number or timestamp to prevent replays. This follows standard QKD guidelines: the classical “distillation channel” must be authenticated. Any message failing verification is discarded, preventing an attacker from injecting fake QBER readings.

Quantum Channel Trust Score (QCTS): We define QCTS as a numeric score (e.g. 0–100) representing link health. A simple formula could be:

$$\text{QCTS}_{\text{new}} = \alpha \cdot \text{QCTS}_{\text{old}} + (1-\alpha) \cdot (100 - 100 \cdot \min(\text{QBER}, \text{QBER}_{\text{th}}) / \text{QBER}_{\text{th}})$$

where `QBER_th` is the acceptable threshold (e.g. 2%), and α ($0 < \alpha < 1$) is a smoothing constant. On each interval, if $\text{QBER} \leq \text{threshold}$, the score moves toward 100; if QBER spikes above threshold (indicating an anomaly or Eve), the score rapidly drops. The router triggers a lockdown if QCTS falls below a cutoff (e.g. 0).

Parameter Tuning: We will tune α and `QBER_th` using the simulated dataset: treating anomaly detection as a signal detection problem, we select values that optimize true positive vs. false positive tradeoff (measured via ROC). For instance, train on part of the dataset to set α (higher α means smoother QCTS, lower sensitivity) and validate on held-out data.

Flowchart: Below is a simplified flow of QCTS computation:

flowchart TD

```
A[Start Session] --> B{Receive QBER Report?}
B -- Yes --> C[Authenticate & Verify]
C --> D{Valid Auth?}
D -- No --> E[Discard Report / Alert]
D -- Yes --> F[Compute New QCTS:  $\alpha \cdot \text{old} + (1-\alpha) \cdot f(\text{QBER})$ ]
F --> G{QCTS < Trigger?}
G -- Yes --> H[Enter Lockdown (shutdown port)]
G -- No --> I[Continue Normal]
B -- No --> I
H --> J[Log Event & Drop Keys]
I --> K[Monitor Next Interval]
```

```
style A fill:#eef,stroke:#333,stroke-width:1px
style H fill:#fbb,stroke:#933,stroke-width:1px
```

This flowchart shows that each received, authenticated QBER reading updates the trust score. On a threshold breach (QCTS low), the router disables the quantum link. Logs record the event for forensics.

6. Formal Verification Targets

We recommend formally modeling the router's finite-state machine (FSM) and verifying key properties:

- **FSM Variables:** states (Normal, Elevated, Lockdown), QCTS value, session counter.
- **Invariants:**
 - *Safety:* "In Lockdown state, the router's QKD port is permanently closed."
 - *Monotonicity:* "QCTS only decreases (or resets) on valid events, never increases spuriously."
 - *Soundness:* "The router does not re-open a port unless QCTS has been high for N consecutive intervals."
- **Liveness/Progress:** If QBER stays below threshold for sufficient time, the system eventually returns to Normal.
- **No Deadlock:** The FSM always allows handling of new telemetry inputs.

Specification Approach: We can encode these properties in **TLA+** or **Spin (Promela)**. For example, in TLA+ one would declare a variable `state` $\in \{\text{Normal, Elevated, Lockdown}\}$ and an invariant `state  $\neq$  Elevated  $\Rightarrow$  port_closed`, etc. We then model steps on receiving QBER and advancing QCTS. The Kim & Kang paper used TLA+ to verify SDN firewall rules; we can similarly check our FSM: e.g. if `QCTS < 0` condition holds, ensure `state=Lockdown` in all reachable states. The TLC model checker or Spin would exhaustively explore possible event sequences to catch logic bugs.

Tools: We suggest **TLA+** (with TLC) for its expressiveness in temporal logic, and **Spin** as an alternative. These are proven in networking contexts (e.g. verifying OpenFlow

rules). A successful verification would provide guarantees such as “the router will never erroneously leave Lockdown until QCTS recovers.”

7. Hardware Constraints for DSL-2750U C1 and Alternatives

The D-Link DSL-2750U C1 is extremely constrained: **4 MB flash and 32 MB RAM**. By Stephen Grier’s account, OpenWrt *can* run on such minimal specs, but it’s at the absolute limit and “not recommended for future use”. In practice, firmware must fit in <3 MB and swap/paging is impossible. All code (TLS libraries, ML model, etc.) must be highly optimized (e.g. use uClibc, dropbear, static linking only if necessary). Resource-heavy features (like an ML model or quantum simulation) may require offloading or a more capable device.

As a reference, many projects use **8 MB flash / 128 MB RAM** routers for modern OpenWrt. For example, the TP-Link Archer C7 (AC1750) has a single-core 720 MHz CPU, 128 MB RAM, and 8 MB flash, which “runs OpenWrt comfortably” (though not at gigabit). Thus a recommended upgrade for ShorlyNot would be a router with ≥ 8 MB/64 MB, or a small single-board computer (Raspberry Pi with OpenWrt) that can handle crypto and ML libraries. GL.iNet routers (e.g. AR750S) or even a low-cost x86 board (LattePanda, Intel NUC) can be used if budget allows. These provide headroom for features like OVS and Python libraries.

In summary, **practical constraints** on the DSL-2750U C1:

- **Flash (4 MB):** Leaves little room. The firmware image must exclude all non-essentials (no WebUI, minimal shell, only needed binaries).
- **RAM (32 MB):** Severely limits concurrency. Feature choice must favor minimal memory footprint (e.g. no Java/Python on the router).

If higher-end hardware is possible, our design scales to it; if stuck with DSL-2750U, features like ML or decoy-state may need to run off-device.

Visualizations

To illustrate key ideas:

*Figure: Conceptual QKD network architecture (from QSI/Aliro). It shows a layered model with **Physical** (quantum channels, repeaters), **Control** (entanglement/key gen), and **Management** layers, matching the common 6-layer QKD design. ShorlyNot operates at the edge of this stack, interfacing classical control with the quantum channel.*

flowchart TD

```
A[Telemetry Reception] --> B{Authentication OK?}
B -- No --> C[Discard/Alert]
B -- Yes --> D[Compute QCTS :=  $\alpha \cdot \text{old} + (1-\alpha) \cdot (100 - \text{scale}(\text{QBER}))$ ]
D --> E{QCTS < Threshold?}
E -- Yes --> F[Lockdown: close port, log event]
E -- No --> G[Normal: keep link open]
F --> H[Optionally trigger reroute]
G --> H
```

*Figure: **QKD Channel Trust Scoring (QCTS) Flow**. Each received QBER report is authenticated; the trust score is updated. If trust falls below the safety threshold, the router enters **Lockdown**, disabling the quantum link. Otherwise the link remains active.*

gantt

```
dateFormat YYYY-MM-DD
title ShorlyNot Feature Development Timeline
section Phase 1: Planning
Requirements & Design           :done, 2026-07-10, 2w
section Phase 2: Implementation
Telemetry & QCTS Engine         :active, 2026-07-24, 3w
Decoy-State Integration         :after 2026-08-14, 3w
ML-Based Detection              :after 2026-09-04, 4w
section Phase 3: Verification
Formal FSM Verification         :after 2026-10-02, 2w
Attack Scenario Testing         :after 2026-10-16, 3w
section Phase 4: Completion
Optimization & Packaging         :after 2026-11-06, 2w
Documentation & Release         :after 2026-11-20, 2w
```

These charts outline the **QCTS logic** and an example **Gantt schedule** for developing ShorlyNot's new features (dates are illustrative). Together with the cited research, they ensure a clear, measurable plan.

Sources: We rely on recent primary literature and technical sources. For example, SDN-based QKD demos, surveys of ML in QKD, QKD networking standards, and OpenWrt documentation/blogs. Citations are given above for specific claims.



iptable.zip

7/5/2026, 11:57:51 PM



Got the `iptable.zip`. I can inspect the **actual codebase**, map what you've already implemented against the ShorlyNot paper, and tell you **what is real, what is weak/fake/demo-only, what research features are missing, and exactly what to build next**.

If that's why you sent it, I'll audit the ZIP directly — no need to explain the project again.

7/5/2026, 11:58:24 PM



everything real right? no fake demo only or simulation?
except for the dashboard?

7/5/2026, 11:59:25 PM



No bro. **I checked the actual `iptables.zip` code. I cannot tell you “everything is real except the dashboard.”**

The current state is roughly **60-70% functional prototype, 30-40% simulated/demo logic.**

The biggest issue: **the quantum layer is simulated.** `quantum_engine/bb84_simulator.py` literally describes itself as a `Lightweight BB84-style QKD simulator`. It uses `Python random.Random`, manually models Alice/Bob bases, Eve interception and a fixed `DEFAULT_NOISE_RATE = 0.01`. **No real quantum hardware, no Qiskit execution, and no actual quantum channel.**

The **AES-GCM encryption is real.** Nonces use `os.urandom(12)`, HKDF-SHA256 is actually called, ciphertext is genuinely encrypted, and the chat/client networking code is functional.

The **KMS is functional code**, but there is a demo hack. It stores `_last_key` and deliberately gives another device the same cached key using what the source itself calls **“shared demo-key semantics.”** The tests even contain comments such as `"Hack"` and `"for the hackathon demo purpose"`. So this is **not proper QKD key agreement or production key distribution yet.**

The **hybrid PQC feature is fake-ish right now.** `pqc_secret = os.urandom(32)` is used. That is **not ML-KEM/Kyber**. You're mixing a random 32-byte secret into HKDF and calling the mode hybrid. Cryptographically the random secret is real entropy, but **claiming this as PQC would be incorrect.**

The **iptables firewall enforcement is real code.** `router_guard.sh` genuinely creates a `QCHAT` chain and executes `iptables -A ... DROP/ACCEPT` if iptables exists.

But **IP switching and network switching are simulated.** The script changes:

```
CURRENT_IP="$BACKUP_IP"
```

and

```
CURRENT_NET="$BACKUP_NET"
```

Those are shell variables. It does **not actually execute** `ip addr` , `ip route` , `uci` , **interface restart, policy routing, or physical uplink switching**. Your own `HOW_IT_WORKS.md` explicitly says Level 2 IP switching and Level 3 network switching are **“simulated/logged.”**

Port rotation is also **partially real**. The firewall changes rules around `CURRENT_PORT` , but I need to verify that the actual chat listener/server is rebound to the new port. From the first audit, it looks like the FSM changes the firewall's idea of the active port rather than performing a coordinated server/client port migration.

The `/link_status` integration is real HTTP polling, but **not securely authenticated**. The router trusts whatever JSON comes from the configured KMS IP. There is no HMAC, signature, sequence validation, or replay protection.

Also, your test suite currently **does not cleanly pass with plain** `pytest -q` . Pytest tries to collect `test_output.txt` and crashes on a UTF-8 decoding error. That's probably test configuration rather than core logic, but I would **not say “all tests pass” right now**.

Current truth

Component	Status
AES-256-GCM	REAL
HKDF-SHA256	REAL
Socket/chat communication	REAL
KMS HTTP service	REAL
QBER calculation	REAL calculation on simulated data
BB84	SIMULATED
Quantum channel	SIMULATED
Eve attack	SIMULATED
iptables DROP/ACCEPT	REAL
Port rotation	PARTIAL
IP failover	SIMULATED
Network/interface switching	SIMULATED

Component	Status
Lockdown firewall	REAL
PQC / ML-KEM	NOT IMPLEMENTED
Hybrid QKD+PQC	FAKE LABEL currently
Shared key distribution	DEMO HACK
Dashboard	DEMO/UI

So **no, don't present the current ZIP as "everything is real except dashboard."** A technical judge who opens the source can catch this very quickly.

The good part is that **most of the fake/demo parts are replaceable without rebuilding the whole project.** We need to fix **5 things: real QKD backend execution or clearly scoped simulator backend, actual ML-KEM, proper key/session agreement, actual network/interface migration, and coordinated port migration.**

After those, you can much more defensibly say:

"All network enforcement, cryptographic protection, key lifecycle, failover, and orchestration mechanisms are functionally implemented. The quantum channel is emulated because physical QKD hardware is unavailable."

That's the version I would build before touching the paper.

7/5/2026, 11:59:28 PM



ok based on reasearch wahat u did and current state how would u suggest me to go what features to add what to do let me know

7/6/2026, 12:18:27 AM



Based on the **research direction + the actual current state of your code + the paper**, I would **not keep adding random quantum features**.

Your paper already claims BB84/QBER, KMS, Toeplitz/HKDF, AES-GCM and a five-state FSM, but it also explicitly admits simulation-only BB84, no reconciliation, no PNS coverage and an unauthenticated router status endpoint. `ShorlyNot_IEEE_Format.docx`

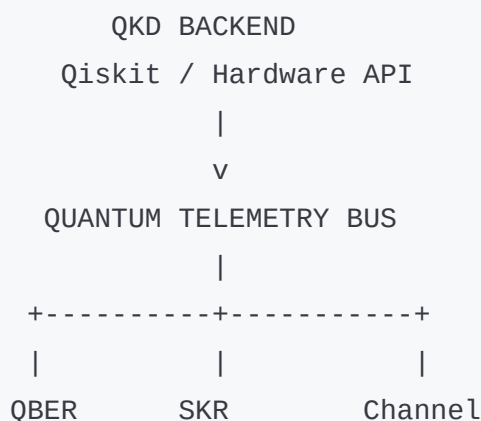
`ShorlyNot_IEEE_Format.docx`

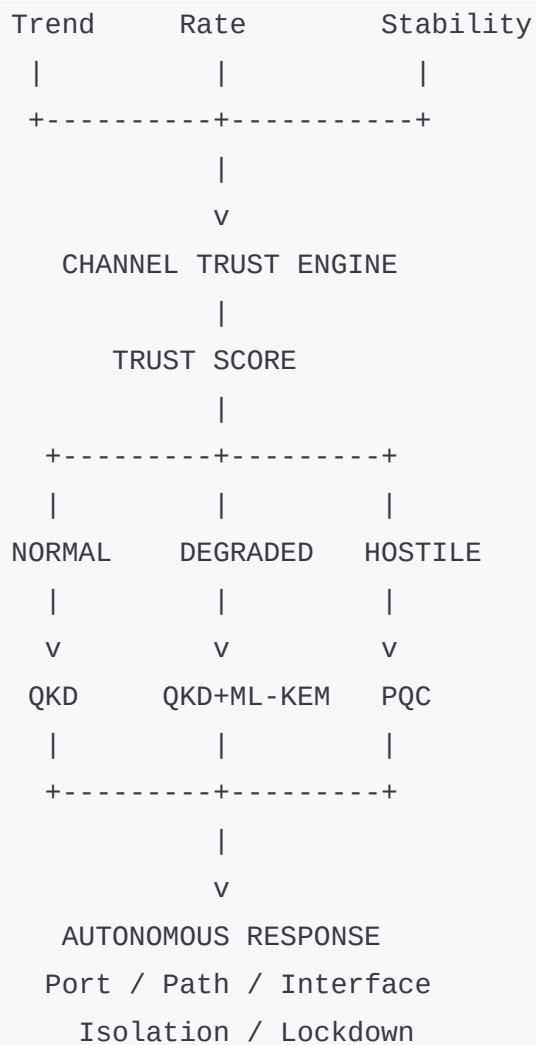
The literature is also moving toward **QKD control/management, dynamic routing, standards-aware key management and QKD-PQC hybrid architectures**, so simply adding “AI predicts Eve” or “we reroute based on QBER” is no longer a safe novelty claim. Recent work already studies SDN-enabled QKD management, real-time link-error/key-aware routing and PQC-secured trusted nodes. ([arXiv](#))

I would rebuild ShorlyNot into this ShorlyNot Sentinel

A quantum-telemetry-driven autonomous security enforcement plane for hybrid QKD-PQC networks.

The architecture I want is:





The **important shift** is:

Current:

```
QBER >= 11%
  ↓
RED
  ↓
FSM++
```

New:

```
Quantum Telemetry
  ↓
Channel Behaviour Analysis
  ↓
Trust Evaluation
  ↓
```

Crypto Mode Selection

↓

Network Enforcement

That is the project.

1. First, make every fake implementation real

Before innovation.

Replace your fake hybrid secret:

```
pqc_secret = os.urandom(32)
```

with actual **ML-KEM-768** based on the [NIST ML-KEM standard](#).

Use [Open Quantum Safe](#) or a suitable ML-KEM implementation.

Then:

```
QKD KEY
|
+-----+
      |
      v
    HKDF
      ^
      |
+-----+
|
ML-KEM SHARED SECRET
```

Derive:

```
SESSION_KEY =
HKDF(
    QKD_KEY ||
    MLKEM_SHARED_SECRET ||
    SESSION_ID
)
```

This gives you **actual hybrid cryptographic composition**.

Don't claim you invented hybrid QKD/PQC. Research already demonstrates QKD systems combined with PQC-secured trusted-node architectures. ([arXiv](#))

Your contribution is **automatic cryptographic mode switching based on quantum-channel telemetry**.

That distinction is important.

2. Build a Quantum Telemetry Protocol

This is probably my favourite addition for your project.

Delete:

```
GET /link_status
```

as the primary router mechanism.

Build:

```
QTP
Quantum Telemetry Protocol
```

A message:

```
{
  "version": 1,
  "node_id": "qkd-node-a",
  "channel_id": "qch-01",
  "session_id": "f81a...",
  "sequence": 9182,
  "timestamp": 1783289212,

  "qber": 0.074,
  "skr": 12842,
  "sifted_key_rate": 24192,
  "key_pool": 8192,

  "backend": "AER_SIMULATOR",

  "signature": "..."
```

The telemetry message gets signed.

Add:

```
sequence number
timestamp
nonce
node identity
signature
replay cache
stale message rejection
out-of-order rejection
```

Research on QKD authentication specifically highlights replay resistance and the complementary role of PQC for QKD authentication. ([arXiv](#))

Now attack it.

```
ATTACK 1
Replay old GREEN QBER

ATTACK 2
Modify QBER 0.25 → 0.03

ATTACK 3
Fake KMS node

ATTACK 4
Delay RED telemetry

ATTACK 5
Reorder telemetry

ATTACK 6
Flood telemetry bus
```

Then show:

Attack	Old ShorlyNot	Sentinel
Replay GREEN	Accepted	Rejected
QBER modification	Accepted	Signature failure
Fake KMS	Accepted	Identity failure

Stale telemetry	Accepted	TTL rejection
Reordering	Accepted	Sequence rejection

This gives you a completely new experiment section.

And it directly fixes the unauthenticated `/link_status` attack surface your paper already admits. `ShorlyNot_IEEE_Format.docx`

3. Build Quantum Channel Trust, but don't use random AI

I changed my view slightly after looking at the literature.

Don't immediately add an LSTM/GNN.

Recent QKD research is already moving toward GNN-based optimization and AI-driven predictive routing. ([arXiv](#))

If you add:

AI predicts quantum attacks 🤖

you will again look like a hackathon project.

Instead build an **interpretable temporal trust engine**.

Monitor:

- QBER
- QBER slope
- QBER variance
- Secure Key Rate
- Sifted Key Rate
- Key pool depletion
- Consecutive anomalies
- Telemetry freshness

Example:

Session	QBER
S1	3.7
S2	3.9
S3	4.2
S4	5.1

S5	6.4
S6	7.8
S7	9.3
S8	10.4

Old ShorlyNot:

NOT RED
NOT RED
NOT RED
NOT RED

Sentinel:

S5 → CHANNEL DEGRADATION DETECTED

Reason:

QBER slope exceeded baseline
variance increased 3.1x
6-session upward trend

Use something academically defensible:

EWMA
+
CUSUM
+
rolling variance

Then compare:

Static 11% Threshold
vs
EWMA
vs
CUSUM
vs
Combined Trust Engine

Metrics:

Detection lead time
False positive rate
Detection probability
Mean time to detect
Threshold sensitivity

This is much better than “we used AI.”

Your current paper uses 11% as a hard operational boundary. `Shor lyNot_IEEE_Format.docx`

Now your research question becomes:

Can temporal quantum-channel telemetry identify channel degradation before a static QBER rejection boundary is crossed?

That is an actual experiment.

4. Build Crypto Agility

This is where I think your project can become genuinely impactful.

Create:

CRYPTO MODE 0
CLASSICAL

CRYPTO MODE 1
ML - KEM

CRYPTO MODE 2
QKD

CRYPTO MODE 3
QKD + ML - KEM

CRYPTO MODE 4
EMERGENCY PQC

The trust engine chooses:

TRUSTED QKD CHANNEL

QKD + AES-GCM

Then:

DEGRADING CHANNEL

QKD + ML-KEM

Then:

QKD REJECTED

ML-KEM only

Then:

KMS / TELEMETRY COMPROMISE

LOCKDOWN

So ShorlyNot no longer says:

QBER RED. Goodbye.

It says:

The quantum channel is no longer trusted. Migrate the security mechanism while maintaining the session's security policy.

That is much more operationally relevant.

QKD and PQC are complementary rather than interchangeable, and published work has explicitly investigated PQC authentication of QKD and hybrid quantum-network deployments. ([arXiv](#))

Your research contribution becomes the **policy and enforcement architecture controlling that migration.**

5. Replace the dumb FSM

Current:

```
NORMAL
↓
PORT
↓
IP
↓
INTERFACE
↓
LOCKDOWN
```

Your paper currently defines exactly this sequential escalation model.

ShorlyNot_IEEE_Format.docx

Problem:

Why must a port change happen before an interface change?

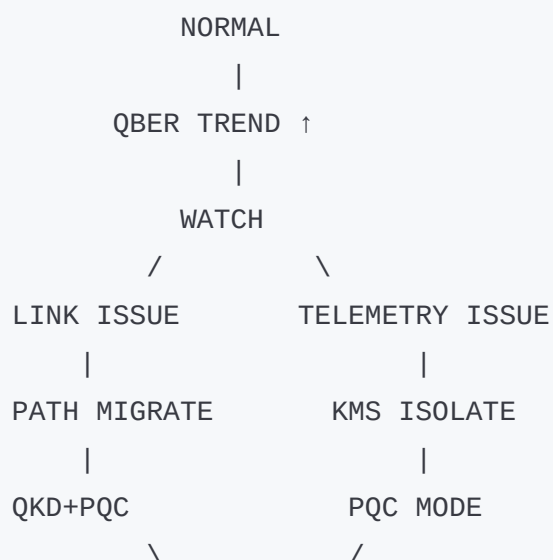
Imagine:

```
eth0 QBER = 18%
wlan0 QBER = 2%
```

Why rotate a port?

That's dumb operationally.

Replace it with a **policy graph**.



RECOVERY

|

NORMAL

Actions are selected from evidence.

QBER spike

→ crypto migration

QBER sustained degradation

→ path migration

Telemetry signature failure

→ isolate KMS

Replay attack

→ reject telemetry source

All paths hostile

→ lockdown

Key pool exhaustion

→ PQC fallback

This is **context-aware orchestration**.

Much stronger than:

```
level += 1
```



6. Create an adversarial Quantum Network Lab

This should replace the random banking use case as a major research component.

Build:

attacks/

├─ full_intercept.py

├─ partial_intercept.py

├─ gradual_intercept.py

```
|— burst_intercept.py
|— noise_mimic.py
|— qber_replay.py
|— telemetry_tamper.py
|— stale_telemetry.py
|— kms_spoof.py
|— key_pool_exhaustion.py
```

Run:

```
10 attack classes
x
10 intensities
x
1,000 sessions
```

Output:

```
100,000 evaluated sessions
```

Generate:

```
QBER distribution
ROC curve
Detection lead-time CDF
False positive curve
Attack confusion matrix
Trust score trajectory
Crypto migration latency
Packet loss during migration
CPU usage
RAM usage
Key consumption
```

Your current evaluation is 500 clean sessions and 50 simulated attack sessions, with intercept-resend as the main evaluated attack. `ShorlyNot_IEEE_Format.docx`

This upgrade alone makes the evaluation **way more serious**.

7. Add a QKD Backend Abstraction Layer

Do this properly.

```
class QuantumKeyBackend:

    def create_session():
        pass

    def get_qber():
        pass

    def get_secure_key_rate():
        pass

    def get_key():
        pass
```

Then:

```
AerBackend
SyntheticChannelBackend
ReplayDatasetBackend
HardwareQKDBackend
```

Architecture:

```
ShorlyNot
|
Quantum Backend API
|
+---+-----+-----+
|           |           |
Aer        Dataset    Hardware
```

This matters because your paper currently claims the quantum engine can later be replaced while leaving the KMS/FSM architecture unchanged. `ShorlyNot_IEEE_Format.docx`

Prove that claim in code.

Don't just write it.

Implement the adapter interface.

Then your paper can say:

The orchestration plane is backend-agnostic and consumes normalized quantum-link telemetry rather than simulator-specific state.

That's clean.

8. TLA+ verify the policy engine

Do this.

Seriously.

Model:

```
NORMAL
WATCH
MIGRATING
PQC_FALLBACK
ISOLATED
LOCKDOWN
```

Verify:

```
RED QKD session never releases QKD key

Invalid telemetry cannot trigger TRUSTED state

LOCKDOWN cannot auto-recover

Session cannot use rejected key

Key ID cannot be reused

Sequence number cannot decrease

At least one secure mode is selected before traffic resumes
```

Formal verification with TLA+ is well suited to state-machine safety properties and dynamic reconfiguration; published work uses TLA+ to machine-check safety properties of distributed reconfiguration and digital-twin FSMs. ([arXiv](#))

This is a **cheap feature with massive paper value**.

You don't need hardware.

You need a `.tla` specification and TLC results.

9. Add a Key Health Plane

This is another thing I think fits ShorlyNot.

Dashboard:

QUANTUM KEY POOL

Available keys	8,192 bits
Consumption rate	512 bit/s
Generation rate	428 bit/s
Estimated depletion	94 sec

STATUS

KEY DEFICIT PREDICTED

Then:

Generation < Consumption

↓

Predict key exhaustion

↓

Enable hybrid mode

↓

Preserve QKD keys for priority traffic

For tactical use:

PRIORITY 1

Command traffic

→ QKD keys reserved

PRIORITY 2

Telemetry

→ Hybrid

PRIORITY 3

Bulk transfer

→ ML-KEM

Dynamic QKD routing literature already treats key-pool availability and link-error rates as important network state, and standards-aware key management/interoperability remains an active systems problem. ([arXiv](#))

Your twist:

security-policy-aware quantum key allocation.

That fits tactical communication extremely well.

What I would NOT build

Don't add:

- blockchain
- federated learning
- GenAI
- LLM SOC agent
- quantum machine learning
- facial recognition
- more dashboards
- another banking website
- "AI Eve prediction"
- satellite simulation just for buzzwords

They will dilute the project.

Your paper already has **two use-case deployments fighting for space**, while its strongest contribution is the KMS/orchestration layer. `Shor lyNot_IEEE_Format.docx`

I would actually **demote the banking app to a demo adapter**.

Exact order I suggest

1. **Fix the demo hacks** - real session agreement, coordinated port migration, actual network migration.
2. **Implement real ML-KEM-768.**
3. **Create the QKD backend abstraction.**
4. **Build signed Quantum Telemetry Protocol.**
5. **Build EWMA/CUSUM temporal channel trust.**
6. **Replace linear FSM with evidence-driven policy orchestration.**
7. **Add automatic QKD ↔ Hybrid ↔ PQC crypto agility.**
8. **Build the 10-attack adversarial test harness.**
9. **Add key-pool health and priority-aware allocation.**
10. **Verify safety properties with TLA+.**
11. **Then add custom OpenWrt firmware as the edge enforcement target.**

The final project I would pitch

ShorlyNot is a quantum-aware security enforcement plane that converts authenticated quantum-link telemetry into real-time trust decisions, cryptographic mode migration, key-resource allocation, and autonomous network response. Rather than treating QBER as a binary alarm, the framework evaluates temporal channel behaviour and securely transitions communication between QKD, hybrid QKD-ML-KEM, and post-quantum fallback modes.

That is the ShorlyNot I think you should build.

Your current version is basically **“BB84 simulator + QBER threshold + policy engine.”**

This version becomes **“an autonomous control plane for operating hybrid quantum-secure networks.”**

That's a much bigger and more defensible project.

Sources

[A new control- and management architecture for SDN-enabled quantum key distribution networks](#)

arxiv.org

7/6/2026, 12:18:29 AM



Quantum-Safe Finance.pdf



Problem Statement Quantathon'...

its for 2 hackathons

1.All that you need to know about IEEE YESIST12 2026 – IEngage Track

The IEEE YESIST12 IEngage Track is a globally recognized, industry-driven innovation challenge that bridges the gap between academia and real-world industry. As part of the prestigious IEEE YESIST12 — Age of Innovation platform, the IEngage Track places industry at the very heart of the innovation journey, encouraging students and young professionals to engage with authentic, industry-curated problem statements and develop solutions that address real market and societal needs.

Aligned with the United Nations Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure), the track focuses on advancing resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering cutting-edge technological innovation across domains.

Mode of Conduct:

This opportunity is conducted fully online. All abstract and project submissions are made through the official IEEE YESIST12 portal (ieeeyesist12.org). Preliminary evaluations and the Grand Finale presentations are conducted virtually via the IEEE YESIST12 platform.

What is the IEngage Track?

IEngage is not just a competition — it is a complete innovation ecosystem built on four foundational pillars: Career Building, Industry Skills Development, Start-up & Entrepreneurship Development, and Industry Mentorship.

The track is designed to nurture industry-ready innovators, foster meaningful collaboration, and drive purpose-driven innovation with measurable industrial and societal impact.

Selected teams, alongside prizes, receive structured guidance from industry professionals through dedicated mentorship cycles, technical sessions, and expert interactions — making the experience far beyond a typical hackathon or academic project.

Industry Problem Statements

Participants choose from real-world, industry-defined problem statements released by leading companies. The current edition features challenges from Capgemini and Tristha:

Capgemini Problem Statements:

AI-Driven Cloud Ops — Build proactive anomaly detection and self-healing cloud systems to reduce MTTR and MTTR across multi-cloud, microservices, and containerized environments.

Quantum-Safe Finance — Design a hybrid classical-quantum security architecture to protect financial systems from emerging quantum threats, including Post-Quantum Cryptography (PQC) evaluation and a phased migration roadmap.

Smart E-Waste Platform — Develop a sustainable, end-to-end digital e-waste tracking and recovery platform for India with EPR compliance and real-time analytics.

Tristha Problem Statements:

AI-Assisted Ticket Classification — Build an intelligent system that classifies support tickets, estimates urgency, and drafts grounded, policy-aligned responses.

AI-Based Duplicate Bug Detection — Develop a system using DBSCAN clustering to detect duplicate defect reports, assign cluster IDs, and generate improved defect summaries.

Grounded Question Answering over Enterprise Documents

— Build a RAG-based Q&A bot that ingests enterprise documents and generates citation-backed answers strictly grounded in approved content.

Topics & Focus Areas:

Artificial Intelligence & Machine Learning — Deep Learning, NLP, Computer Vision, AI in real-world problem solving
Cloud Computing & DevOps — AWS, Azure, Google Cloud, Kubernetes, CI/CD pipelines, Infrastructure as Code
Cybersecurity — Ethical hacking, network security, penetration testing, incident response

Data Science & Analytics — Data visualization, statistical modeling, Big Data, Business Intelligence

Entrepreneurship & Startup Development — Business model canvas, pitch decks, funding strategies, product-market fit

Soft Skills & Professional Development — Communication, leadership, time management, interview preparation

Who Can Participate?

The IEngage Track is open to:

Undergraduates — Students currently pursuing undergraduate degrees

Postgraduates — Students pursuing master's or doctoral programs

Young Professionals — Early-career professionals seeking real industry exposure

Team Requirements:

Teams must consist of a minimum of 2 and a maximum of 4 members

Teams can be from the same institution or multi-institutional collaborations

A mandatory mentor (Faculty, Industry Professional, or IEEE Member) must be assigned to each team

Selection Process

The selection follows a two-stage fully online process:

Stage 1 — Abstracts & Prelims: Participants enter via Direct

Entry (abstract evaluated online by reviewers) or Pilot Entry (preliminary round conducted virtually by IEEE Sections or partner institutions, with winners advancing to finals).

Stage 2 — Grand Finale: Shortlisted teams present their solutions virtually before an expert jury, evaluated on novelty, technical implementation, clarity of pitch, and expandability of the idea.

Why Should You Participate?

Work on real, industry-defined challenges that go beyond routine academic projects

Get mentored by experienced industry professionals and domain researchers

Develop hands-on skills in ideation, solution design, feasibility analysis, and professional presentation

Compete with high-performing teams from across the globe

Present your solution at an internationally recognized IEEE event

Earn IEEE-recognized certificates, awards, and international recognition

Connect with industry leaders and open doors to internships and career collaborations

Be part of a unique innovation journey that blends competition, mentorship, learning, and real-world impact

Rules for Participation:

All participants must register through the official IEEE YESIST12 portal (ieeeyesist12.org)

Regular attendance at online workshops and sessions is mandatory for certification

All assignments and tasks must be completed within given deadlines

Participants must adhere to the IEEE Code of Conduct at all times

Plagiarism is strictly prohibited in all submissions

The IEngage Track is your opportunity to step beyond the classroom, prove your skills on a global stage, and make a real impact through innovation. Whether you're solving AI challenges, designing secure financial systems, or building sustainable platforms — IEngage gives you the tools,

mentorship, and platform to turn your ideas into industry-ready solutions.

All rounds are conducted fully online via ieeeyesist12.org

About IEngage Track

As we experience an exceedingly evolving industrial and technological landscape, the IEEE YESIST12 IEngage Track emerges as a platform that bridges innovation with real-world industry challenges. By placing industry at the center of the innovation journey, the IEngage Track encourages participants to engage with authentic problem statements and develop solutions aligned with real market and societal needs. Aligned with the United Nations Sustainable Development Goal 9: Industry, Innovation, and Infrastructure, the track focuses on advancing resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering technological innovation. Selected teams alongside prizes also receive guidance from industry professionals through dedicated mentorship cycles, technical sessions, and expert interactions. The track is not solely about developing solutions, but about nurturing industry-ready innovators, fostering collaboration, and creating purpose-driven innovation with measurable industrial and societal impact.

IEngage Track Image

Current Theme

Industry Focused Innovative and Sustainable Solutions
Launching Soon | What Awaits?

We're gearing up to release industry-curated themes exploring transformative technologies — from agentic AI frameworks to next-gen IoT.

SDGs Aligned

This track encourages projects addressing the following United Nations Sustainable Development Goal:

SDG 9: Industry, Innovation, and Infrastructure

Timeline

Call for Abstracts 15th March 2026

Deadline for Abstract 6th April 2026

Time for reviewers to review the abstracts 27th March – 17th April 2026

Announcement of qualified abstracts 22nd April 2026

Final Call for Abstracts 27th April 2026

Deadline for Final Call for Abstract 19th May 2026

Time for reviewers to review the abstracts 2nd May – 20th May 2026

Announcement of qualified abstracts 22nd May 2026

IEEE YESIST12 IEngage Track Problem Statements:

Capgemini Logo

Industry Problem Statements

1. AI-Driven Cloud Ops (Proactive Anomaly Detection & Self-Healing Cloud Systems)

This problem statement addresses the limitations of current cloud operations models that rely heavily on manual log analysis, rule-based alerting, and human-driven incident response. As cloud environments scale across multi-cloud, microservices, and containerized architectures, fragmented telemetry and lack of automated remediation increase Mean Time to Detect (MTTD) and Mean Time to Resolve (MTTR).

Key Focus Areas:

AI-driven anomaly detection across logs, metrics, traces, and security telemetry.

Cross-domain event correlation and intelligent incident classification.

Automated remediation workflows and self-healing mechanisms.

Reduction of operational, compliance, and security risks.

For complete technical scope, objectives, constraints, KPIs, and expected outcomes, refer to the detailed document below:

Download Detailed Scope (AI-Driven Cloud Ops)

2. Quantum-Safe Finance (Hybrid Classical-Quantum Security Architecture)

The rapid advancement of quantum computing poses a significant threat to traditional cryptographic systems such as RSA and ECC, which secure financial transactions and sensitive data. Algorithms like Shor's and Grover's may

enable future decryption of currently encrypted data ('harvest-now, decrypt-later' attacks).

Key Focus Areas:

Assessment of quantum threats to financial systems and encryption standards.

Evaluation of Post-Quantum Cryptography (PQC) algorithms.

Design of hybrid classical + quantum-resilient security architecture.

Development of a phased migration roadmap for financial institutions.

For complete methodology, architecture design, strategic outputs, and implementation roadmap, refer to the detailed document:

[Download Detailed Scope \(Quantum-Safe Finance\)](#)

3. Smart E-Waste Platform (Sustainable E-Waste Tracking & Recovery Platform for India)

India is the third-largest producer of e-waste globally, yet lacks a transparent, technology-driven, end-to-end digital ecosystem that ensures traceability, accountability, and incentivized participation across the e-waste value chain.

Key Focus Areas:

Digital tracking of electronic products from manufacturing/import to disposal.

Integration of formal and informal recycling sectors.

Compliance with Extended Producer Responsibility (EPR) regulations.

Real-time analytics for regulators and consumer incentive mechanisms.

For detailed regulatory context, constraints, KPIs, and expected outcomes, refer to the detailed document:

[Download Detailed Scope \(Smart E-Waste Platform\)](#)

Tristha Logo

Industry Problem Statements

1. AI-Assisted Ticket Classification, Urgency Estimation, KB Retrieval, and Grounded Response Drafting

This problem focuses on building a system that classifies

ticket category (Refund, Login, Delivery, Billing, Account, or Other), estimates urgency (High, Medium, or Low), retrieves relevant Knowledge Base snippets, and drafts a grounded reply. The system ensures that draft replies are based on approved content and asks for clarification instead of guessing when information is missing.

Key Focus Areas:

Automated ticket categorization and urgency estimation.

KB ingestion and semantic search for relevant snippet retrieval.

Generation of grounded draft replies with source citations.

Reduction of agent triage time while ensuring professional, policy-aligned responses.

For complete technical scope, objectives, constraints, KPIs, and expected outcomes, refer to the detailed document below:

Download Detailed Scope (AI-Assisted Ticket Classification)

2. AI-Based Duplicate Bug Detection, Defect Clustering, Missing Field Analysis, and Report Improvement

Software teams often receive multiple defect reports describing the same underlying issue. Duplicate defects increase triage effort, fragment engineering discussions, and delay resolution. This problem focuses on building a system that detects duplicate or possible duplicate defect reports using similarity search and clustering, assigns cluster IDs, suggests missing fields, and generates an improved defect summary while marking duplicates only when similarity exceeds a defined threshold.

Key Focus Areas:

Duplicate defect detection and similarity-based clustering (DBSCAN/Nearest-cluster).

Missing field analysis (reproduction steps, logs, environment, etc.) for report enhancement.

Embedding generation and vector database based similarity search.

Decisioning into duplicate, possible_duplicate, or new_defect based on thresholds.

For complete technical scope, objectives, constraints, KPIs,

and expected outcomes, refer to the detailed document below:

Download Detailed Scope (AI-Based Duplicate Bug Detection)

3. Grounded Question Answering over Enterprise Documents using Retrieval-Augmented Generation
Organizations maintain large collections of policies, FAQs, manuals, and operational documents. This problem focuses on building a Retrieval-Augmented Generation (RAG) based Q&A bot that ingests documents, retrieves relevant sections, and generates concise answers strictly from retrieved content with citations. If the answer is not present, the system must avoid guessing and ask for the relevant document.

Key Focus Areas:

Document ingestion, chunking, and preprocessing for effective retrieval.

Embedding generation and semantic search using vector databases.

Answer generation strictly grounded in retrieved content with citations (source, snippet, score).

Fallback handling for unsupported questions to prevent hallucinations.

For complete technical scope, objectives, constraints, KPIs, and expected outcomes, refer to the detailed document:

Download Detailed Scope (Grounded Question Answering)
Objectives

Industry-Driven Innovation: Foster innovation by encouraging participants to develop solutions aligned with real-world industrial challenges and emerging technological needs.

Academia-Industry Bridge: Provide students and young professionals with exposure to practical problem-solving, professional standards, and industry expectations.

SDG 9 Alignment: Promote innovation aligned with UN SDG 9, focusing on sustainable industrial growth and resilient technological systems.

Expert Mentorship: Enable hands-on learning through

direct interaction with industry experts and technical mentors throughout the innovation lifecycle.

Scalable & Feasible Solutions: Encourage development of impact-driven solutions that can be translated into real-world applications, pilots, or collaborations.

Global Visibility: Provide a global platform to showcase ideas, receive expert feedback, and gain international visibility within the IEEE ecosystem.

Talent Identification: Connect industries with high-potential innovators for mentorship, internships, and long-term engagement opportunities.

Sustainable Ecosystem: Emphasize continuity beyond competition, enabling teams to refine solutions through post-event mentorship and guidance.

Eligibility

Undergraduates - Students currently pursuing undergraduate degrees.

Postgraduates - Students pursuing master's or doctoral programs.

Young Professionals - Early-career professionals seeking industry exposure.

Team Requirements

Participants must register as a team.

Each team must consist of a minimum of 2 and a maximum of 4 members.

Teams may be from the same institution, or multi-institutional collaborative teams.

Mandatory Mentor: Faculty, Industry Professional, or IEEE Professional Member.

Why Should You Participate?

Beyond Classroom Projects: Work on real industry-defined challenges that go far beyond routine academic or college-level projects.

Real Industry Readiness Assessment: Test your skills, ideas, and problem-solving approach in an environment aligned with actual industry expectations.

Mentorship from Experts: Selected teams receive structured guidance from experienced industry professionals and domain researchers.

Skill Development: Develop hands-on skills in ideation, solution design, feasibility analysis, and professional

presentation.

Global Competition Exposure: Compete alongside high-performing teams from different countries, institutions, and technical backgrounds.

Showcase Internationally: Present your solution at an internationally recognized IEEE event with global visibility.

Recognition & Certification: Gain international recognition and certificates associated with a prestigious IEEE innovation challenge.

Career Opportunities: Connect with industry leaders and mentors, opening doors to internships and collaborations.

Industry-Oriented Experience: Participate in a structured challenge that strengthens your professional profile and resume.

Unique Innovation Journey: Be part of an experience that blends competition, mentorship, learning, and real-world impact.

Selection Method

The selection process will be conducted in two main stages:

Stage 1 - Abstracts & Prelims

Abstract submission is open to all students via Pilot entry or Direct Entry.

A) Direct entry participants will be evaluated based on abstract submission. Selected innovative projects will be informed via email.

B) Pilot entry participants present projects in a preliminary round conducted by pilots. Winners move to finals.

Stage 2 - Grand Finale

Selected teams will present their projects before the jury during the finals.

Evaluation will be carried out based on criteria such as novelty, technical implementation, clarity of pitch, and expandability of the idea.

Rules for participation

Registration: All participants must complete the registration

process through the official portal.

Attendance: Regular attendance in workshops and sessions is mandatory for certification.

Assignments: Participants must complete all assigned tasks and projects within the given deadlines.

Code of Conduct: All participants must adhere to the IEEE code of conduct and maintain professional behavior.

Plagiarism: Strictly prohibited in all submissions and assignments.

Updates: Workshop schedules and session details will be updated regularly on the website.

Track Structure - The Four Pillars

IEngage Track is built on three foundational pillars that empower participants to excel in their professional journey:

Pillar 1: Career Building

We empower young professionals to build a strong foundation for their careers by equipping them with in-demand technical skills and allow them to demonstrate with innovations.

Pillar 2: Industry Skills Development

Embrace continuous learning and develop industry knowledge to stay relevant and competitive in today's fast-paced technological landscape.

Pillar 3: Start-up/Entrepreneurship Development

IEngage provides right guidance with the three stages of startup namely early-stage, venture-funded (growth) stage and late stage. IEngage unlocks your entrepreneurial potential and fuels your confidence for success.

Pillar 4: Industry Mentorship

Connect with industry experts who provide personalized guidance and insights to help you navigate your career path successfully.

Topics & Focus Areas

Artificial Intelligence & Machine Learning

Explore the fundamentals and advanced concepts of AI/ML, including:

Deep Learning and Neural Networks
Natural Language Processing
Computer Vision applications
AI in real-world problem solving
Cloud Computing & DevOps
Master cloud technologies and modern development practices:

AWS, Azure, and Google Cloud platforms
Containerization and Kubernetes
CI/CD pipelines
Infrastructure as Code
Cybersecurity
Learn essential cybersecurity skills for the digital age:

Ethical hacking and penetration testing
Network security fundamentals
Security best practices
Incident response and management
Data Science & Analytics
Develop data-driven decision making skills:

Data visualization and storytelling
Statistical analysis and modeling
Big data technologies
Business intelligence tools
Entrepreneurship & Startup Development
Build the foundation for your entrepreneurial journey:

Business model canvas and lean startup methodology
Pitch deck creation and presentation skills
Funding strategies and investor relations
Product-market fit and scaling strategies
Soft Skills & Professional Development
Enhance your professional toolkit:

Communication and presentation skills
Leadership and team management
Time management and productivity
Interview preparation and career guidance

Key Focus Areas

Career Development: Build a strong foundation for your professional career with industry-relevant skills.

Industrial Skills: Develop technical competencies that are in high demand in today's job market.

Startup Ecosystem: Learn the fundamentals of entrepreneurship and startup development.

Industry Connections: Build valuable connections with professionals and mentors.

Hands-on Training: Gain practical experience through project-based learning.

Recognition: Receive certificates and recognition for your achievements.

2. quantathon

36-Hour International Live Hackathon · Quantum Technology

QUANT-A-THON '26

Innovate. Build. Transform.

Hosted by The Grover Center for Quantum Technology RIT Chennai , In Association with : IBM | IITM CDOT Samgnya Technologies Foundation | National Quantum Mission
30-31 July 2026

Venue : RIT, Chennai, India.

Teams of

1-4

24

Days

08

Hours

33

Minutes

30

Seconds

Register your team

→

Explore domains

₹5,00,000 cash prize pool · International university internships for winners

Last Date for Registration is 05/07/2026 ♦ Last Date for Registration is 05/07/2026 ♦ Last Date for Registration is 05/07/2026 ♦ Last Date for Registration is 0752026 ♦ Last Date for Registration is 05/07/2026 ♦

Hack the future



Power the Quantum age



Innovate



Build



Transform



Rewards

Win big. Go global.

₹5,00,000

Cash prize for winners

International Internships

Winning teams unlock internship opportunities at international universities.

Industry Mentorship

Direct access to mentors from IBM, the National Quantum Mission and IITM CDOT Samgnya Technologies Foundation.

Certificates & Recognition

Official certificates, national visibility and a place on the winners' wall.

Quantum Platform Access

Hands-on time with leading Quantum computing platforms during the event.

About the challenge

Where India's builders meet the Quantum age.

Quant-A-Thon 2026 is a 36-hour international live hackathon on Quantum technology, hosted by the Rajalakshmi Institute of Technology in association with IBM, National Quantum Mission and IITM CDOT Samgnya

Technologies Foundation.

Over two days, student innovators from across the country — and beyond — form teams to prototype real solutions across Quantum computing, Quantum-powered AI and Quantum cybersecurity, mentored by industry leaders and competing for a ₹5,00,000 prize pool and international university internship opportunities.

₹5,00,000

Cash prize pool

36h

Non-stop hacking

1–4

Members per team

Participants

UG · PG · Ph.D

Build across

Eight frontiers of Quantum.

Pick a domain, assemble your team and ship a working prototype. Each track is backed by mentors from our partner network.

01

Quantum Computing

Design circuits and algorithms on real Quantum hardware and simulators. Tackle optimisation, simulation and speed-up problems.

02

Quantum + AI

Fuse machine learning with Quantum methods — Quantum kernels, hybrid models and accelerated training for tomorrow's intelligence.

03

Quantum Cybersecurity

Build post-Quantum cryptography, secure key distribution and threat models that hold up in a world of Quantum adversaries.

04

Quantum Communication

Secure the future of connectivity with Quantum-enabled networks leveraging entanglement and Quantum cryptography for ultra-secure information exchange.

05

Quantum Neural Networks

Reimagine AI with Quantum-enhanced neural architectures, combining qubits and machine learning to unlock powerful new learning capabilities.

06

Quantum Optimization

Solve complex optimization challenges faster by harnessing quantum algorithms to discover optimal solutions across industries and applications.

07

Quantum Chip Design

Engineer the core of Quantum technology, designing advanced qubit-based processors that power next-generation Quantum computing systems.

08

Quantum Sensing

Detect the invisible with Quantum-enhanced precision, using the unique properties of Quantum states to measure tiny changes in motion, magnetic fields, and the environment.

The 36 hours

Two days. One sprint.

01

Thu · 30 July 2026

08:30

Check-in & Registration

Teams arrive, set up and power on.

10:00

Opening Ceremony + Keynote

Partners set the stage for the Quantum age.

11:00

Hacking Begins at 11.00

The clock starts. Build relentlessly.

21:00

Mentor Round + Midnight Fuel

Check-ins, food and overnight momentum.

02

Fri · 31 July 2026

09:00

Mid-Hack Review

Final guidance before the home stretch.

11:00

Project Demos & Judging

Pitch to the panel of Quantum experts.

18:00

Jury Evaluation

Projects evaluated by an expert jury panel.

19:30

Closing & Award Ceremony

Winners crowned — ₹5,00,000 on the line.

// Indicative schedule — final timings shared with
registered teams.

In association with

Powered by Quantum pioneers.

RIT partners

RIT partners

RIT partners

RIT partners

RIT partners

RIT partners

RIT partners

RIT partners

KwantumG.webp
National Quantum Mission
RIT partners
Ready to power the Quantum age?
Assemble a team of 1-4, pick your frontier and build something the world hasn't seen. Registration is open to UG, PG and PhD students, pan-India and international. check out 3.5

7/6/2026, 12:28:18 AM



Yeah - **now that I have the exact two problem statements, I would change my previous recommendation a bit.**

I checked the uploaded IEEE YESIST12 2026 Quantum-Safe Finance scope and **QT-3.5 specifically** from Quant-A-Thon '26.

The important thing is: **do not build two projects. Build one core system with two competition modes.**

The YESIST12 problem explicitly wants quantum-threat assessment, PQC evaluation, a hybrid classical + quantum-safe architecture, QKD feasibility, financial workloads, and a phased migration roadmap. [Quantum-Safe Finance.pdf](#)

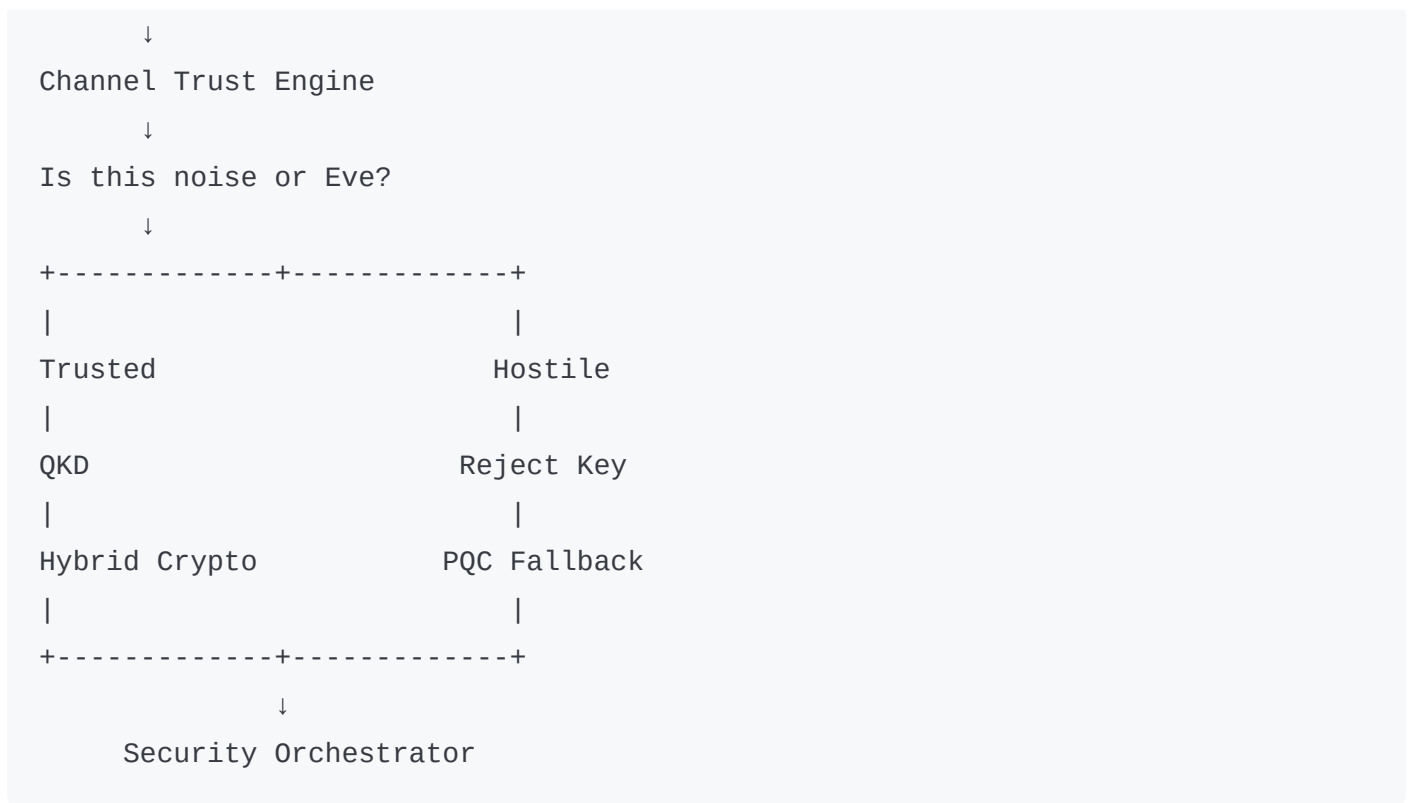
QT-3.5 is much narrower: implement BB84, establish a shared key, detect Eve through QBER, demonstrate secure and compromised scenarios, analyse eavesdropping impact, and provide reproducible code. Its stretch goal is specifically to add realistic channel noise and distinguish natural errors from deliberate eavesdropping. [Problem Statement Quantathon'26.pdf](#)

So this is what I think ShorlyNot should become

ShorlyNot - Quantum-Safe Adaptive Security Orchestrator

Core idea:

```
Quantum Channel
    ↓
BB84 QKD Engine
    ↓
Quantum Telemetry
QBER + SKR + Key Rate
```

This single architecture can hit **both hackathons**.

For Quant-A-Thon: focus almost entirely on QT-3.5

The judges literally give you the scoring:

30% protocol correctness.

25% eavesdropper detection.

20% security analysis and QBER interpretation.

15% reproducibility/code quality.

10% presentation. [Problem Statement Quantathon'26.pdf](#)

So please **do not walk into Quant-A-Thon talking for 10 minutes about ML-KEM, finance migration and TLA+.**

You'll dilute your score.

Your Quant-A-Thon version should be:

A noise-aware BB84 QKD simulator that distinguishes natural quantum-channel degradation from deliberate eavesdropping and autonomously protects compromised key sessions.

That is exactly aligned with their stretch goal.

The killer feature should be this

Observed QBER = 8.4%

Normal BB84 simulator:

QBER < 11%

SAFE

ShorlyNot:

QBER = 8.4%

Historical baseline = 2.1%

QBER slope = +1.7% / session

Variance = 3.4× baseline

Burst correlation detected

CHANNEL STATE:

SUSPICIOUS

Probable cause:

GRADUAL INTERCEPTION

Then another:

QBER = 8.4%

Channel noise baseline = 7.9%

Variance stable

No directional QBER trend

Noise model correlation = 0.91

CHANNEL STATE:

NOISY

Probable cause:
NATURAL CHANNEL DEGRADATION

Fam. That is literally their stretch goal.

The problem statement says:

develop methods to distinguish naturally occurring transmission errors and deliberate eavesdropping attempts based on observed QBER.

You are not bending the PS.

You're attacking the hardest part of it. `Problem Statement Quantathon'26.pdf`

For Quant-A-Thon, I would build only these features

1. Correct BB84 engine in Qiskit.

Alice bits, Alice bases, Bob bases, sifting, sample disclosure, QBER estimation and final shared key verification.

2. Multiple Eve models.

Full Intercept-Resend
Partial Intercept
Gradual Intercept
Burst Intercept
Adaptive Eve

3. Realistic noise models.

Bit flip
Phase flip
Depolarizing
Amplitude damping
Readout error
Loss approximation

4. Noise vs Eve classifier.

Not QML.

Start with:

```
EWMA
CUSUM
Rolling Variance
QBER slope
Basis-specific error distribution
Temporal burst score
```

5. Channel states.

```
CLEAN
NOISY
SUSPICIOUS
COMPROMISED
```

6. Attack replay laboratory.

```
Select Attack:
[ Gradual Eve ]

Interception Rate:
[ 0% ————— 100% ]

Channel Noise:
[ 0% ————— 20% ]

Run 1000 Sessions
```

Output:

```
Eve Detection Rate
False Positive Rate
Mean Detection Time
QBER Distribution
```

Trust Score
Final Key Acceptance Rate

7. Key rejection enforcement.

If compromised:

```
RAW KEY
  ↓
SIFT
  ↓
QBER / TRUST ANALYSIS
  ↓
COMPROMISED
  ↓
KEY DESTROYED
```

The compromised key **must never enter your KMS**.

That's your Quant-A-Thon project.

My Quant-A-Thon pitch

Traditional BB84 demonstrations treat QBER as a static threshold. ShorlyNot studies QBER as temporal quantum-channel telemetry, distinguishing natural channel noise from progressive interception behaviour and autonomously rejecting compromised key sessions before they are used for secure communication.

That is much better aligned than my previous giant Sentinel architecture.

YESIST12 is where we unleash the full ShorlyNot

This is a [Capgemini](#) industry problem.

Their document is basically screaming:

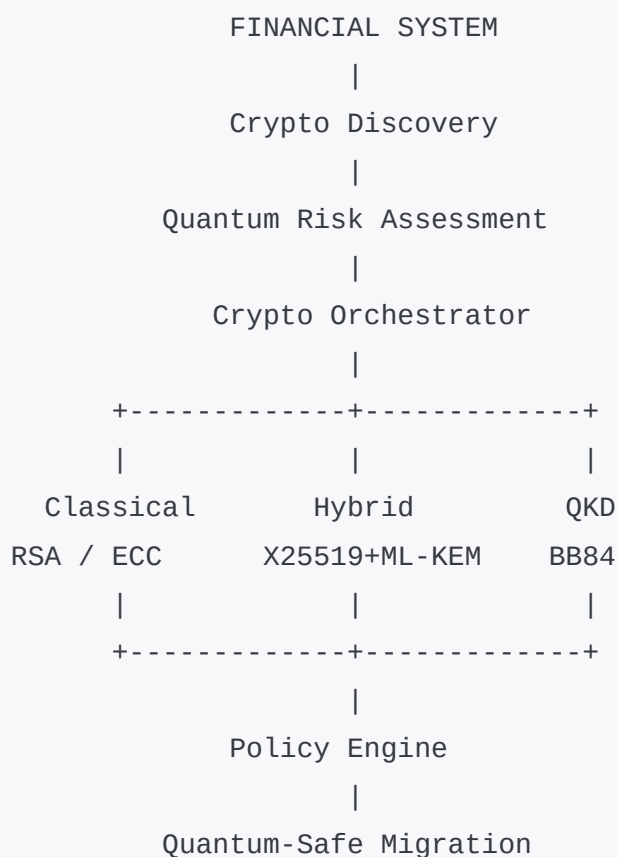
```
PQC evaluation
+
hybrid architecture
+
legacy migration
```

+
financial workload
+
roadmap

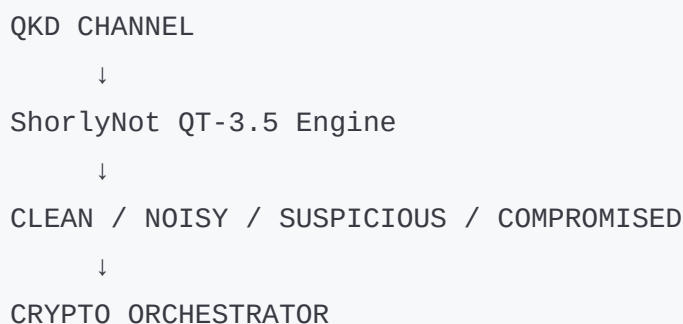
The scope specifically includes PQC applicability, hybrid classical + quantum-safe architecture, QKD feasibility, performance/scalability/interoperability comparisons and an industry-ready transition roadmap. [Quantum-Safe Finance.pdf](#)

Here, **BB84 is only one subsystem.**

Architecture:



And ShorlyNot's QKD engine from Quant-A-Thon plugs in here:



Then:

CLEAN

→ QKD-derived session key

NOISY

→ Hybrid QKD + ML-KEM

SUSPICIOUS

→ ML-KEM fallback

COMPROMISED

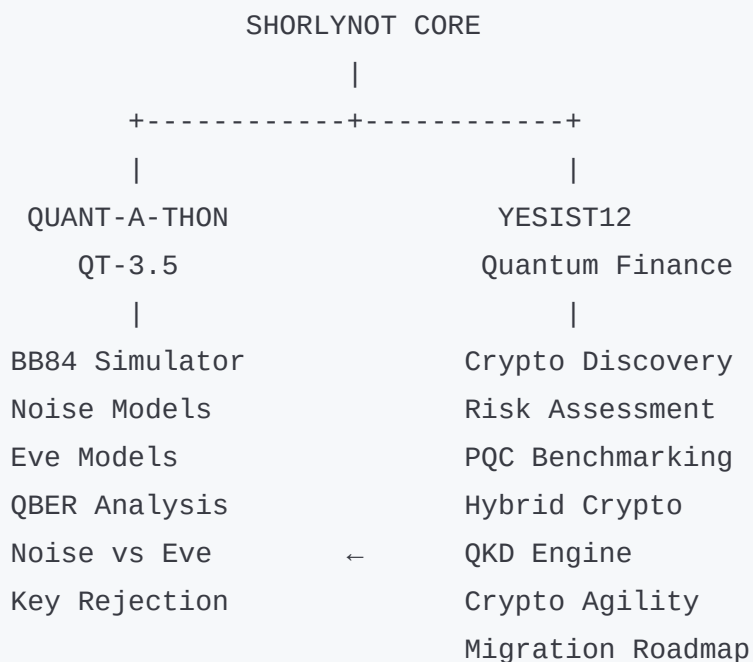
→ Reject QKD key + isolate channel

For YESIST12 add these features

- **Crypto Asset Discovery:** scan certificates, configs and code for RSA, ECC, TLS algorithms and key sizes.
- **Quantum Risk Score:** score assets using algorithm, data retention period, exposure, asset criticality and HNDL risk.
- **PQC Benchmark Lab:** compare ML-KEM and signature schemes on financial-style workloads.
- **Hybrid TLS / key exchange demo:** classical + ML-KEM migration mode.
- **ShorlyNot QKD Engine:** reuse the complete QT-3.5 work.
- **Crypto Agility Engine:** choose Classical → Hybrid → PQC → QKD-assisted modes.
- **Migration Planner:** generate Phase 1 inventory, Phase 2 hybrid pilot and Phase 3 PQC rollout; this directly mirrors the phased strategy in the provided scope. [Quantum-Safe Finance.pdf](#)
- **Financial demo:** payment API, identity/authentication service, inter-bank communication and archival data.
- **Migration Impact Simulator:** show latency, key/certificate growth, bandwidth, storage and compatibility impact.

The YESIST12 document explicitly expects prototype/simulation evidence, PQC evaluation, hybrid models, QKD feasibility and future-ready cryptographic management tools. [Quantum-Safe Finance.pdf](#)

The clean strategy



Quant-A-Thon builds the QKD intelligence engine.

YESIST12 turns that engine into one component of an industry quantum-safe migration platform.

My recommendation for you, Akash

For the next few weeks, **80% focus on QT-3.5**.

Build the BB84 engine properly and make the **noise-vs-Eve distinction your main research contribution**.

Do **not** add ML-KEM yet.

Do **not** add TLA+ yet.

Do **not** add OpenWrt yet.

Do **not** build the massive autonomous quantum network I suggested earlier yet 🤖

Finish:

```
BB84
↓
Noise simulation
↓
5 Eve attack patterns
```


↓
Temporal QBER telemetry
↓
EWMA + CUSUM
↓
Noise vs Eve classification
↓
Key rejection
↓
100,000 session experiment
↓
Classical threshold vs ShorlyNot comparison

Then **freeze that module**.

After Quant-A-Thon, wrap it with:

Crypto Inventory
+
Quantum Risk Scoring
+
PQC Benchmarking
+
ML-KEM Hybrid Mode
+
Migration Planner

for YESIST12.

This is actually a very lucky overlap. QT-3.5 is basically giving you the QKD subsystem that the YESIST12 finance architecture asks you to assess at pilot/feasibility level. The finance scope explicitly keeps large-scale QKD out of scope while allowing conceptual or pilot-level QKD, which fits this reuse perfectly. [Quantum-Safe Finance.pdf](#)

If you want, I can next give you the **final ShorlyNot architecture + exact feature checklist split into CORE / QUANTATHON / YESIST12**, based on what you've already built.

7/6/2026, 12:28:21 AM



give me the whole prd of what to build additionally this should be a guranteed hackathooon winner project plaese give your 100% no simulation no demo no bs i want it to use 100% real quantum implementations

7/6/2026, 12:32:17 AM



Akash, one correction before the PRD: **I cannot guarantee a hackathon win**, and I will not write a PRD that hides simulation behind the word "quantum."

For QT-3.5, the official problem explicitly permits quantum circuit simulators, but you asked for **no simulation**. [Problem Statement Quantathon'26.pdf](#) The YESIST12 scope also explicitly treats QKD as feasibility/pilot-level and excludes large-scale QKD implementation. [Quantum-Safe Finance.pdf](#)

Also, a real BB84 link requires an optical quantum communication setup. A cloud gate-model QPU is **real quantum hardware**, but running BB84-like circuits on it is **not the same as physical photon transmission between Alice and Bob**. Recent experimental QKD systems use actual optical hardware, weak coherent states, detectors and fibre channels. ([arXiv](#))

So the build below follows one rule:

If it is quantum, it runs on real quantum hardware or a real physical quantum source. If we do not have physical QKD hardware, we explicitly call that part an emulated channel-not physical QKD.

Here is the PRD I would actually build.

:::writing{variant="document" id="61482"}

PRODUCT REQUIREMENTS DOCUMENT

SHORLYNOT Q-ORCHESTRA

Quantum-Native Cryptographic Agility and Autonomous Security Orchestration Platform

Version: 3.0

Project Type: Quantum Cybersecurity / Quantum Communication / Financial Infrastructure Security

Primary Competitions: IEEE YESIST12 2026 IEngage - Quantum-Safe Finance and Quant-A-Thon '26 - QT-3.5

Core Principle: No fake quantum claims. No random-number placeholders presented as PQC. No dashboard-only features. Every claimed security action must execute.

1. PRODUCT VISION

ShorlyNot Q-Orchestra is a cryptographic control plane that observes quantum-channel and cryptographic telemetry, evaluates channel and asset trust, and autonomously selects, migrates and enforces the appropriate security mechanism.

The system supports:

- physical quantum entropy acquisition where available;
- real quantum processor execution for quantum measurement experiments;
- BB84 protocol processing;
- QBER analysis;
- authenticated quantum telemetry;
- ML-KEM key establishment;
- hybrid key derivation;
- cryptographic asset discovery;
- quantum-risk assessment;
- automatic cryptographic migration;
- key lifecycle management;
- network enforcement;
- financial workload protection.

ShorlyNot does not claim to invent BB84, QKD, ML-KEM or hybrid cryptography.

The proposed contribution is the integration of quantum-link telemetry, cryptographic inventory state and runtime security policy into an autonomous cryptographic orchestration

plane.

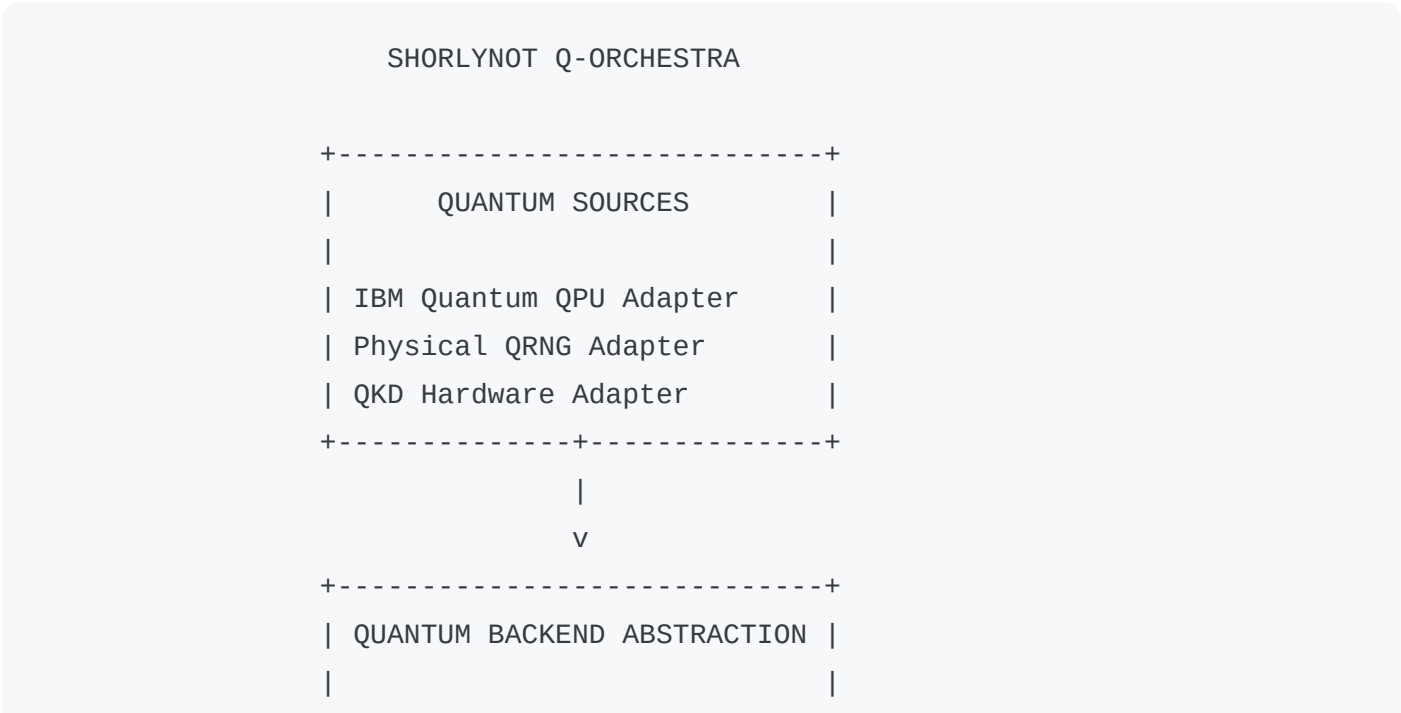
2. PRIMARY RESEARCH QUESTION

Can authenticated quantum-channel telemetry and cryptographic asset intelligence be used to automatically select, migrate and enforce cryptographic protection while maintaining secure operational continuity?

Secondary research questions:

- 1. Can temporal QBER behaviour provide earlier warning than a static QBER threshold?
- 2. Can natural channel degradation be distinguished from progressive interception patterns using interpretable statistical telemetry?
- 3. Can a communication session automatically migrate from a QKD-derived key mode to hybrid or ML-KEM mode without using a rejected quantum key?
- 4. Can financial cryptographic assets be automatically discovered, risk-ranked and mapped to a phased post-quantum migration strategy?
- 5. What latency, memory, bandwidth and key-size overhead is introduced by cryptographic agility?

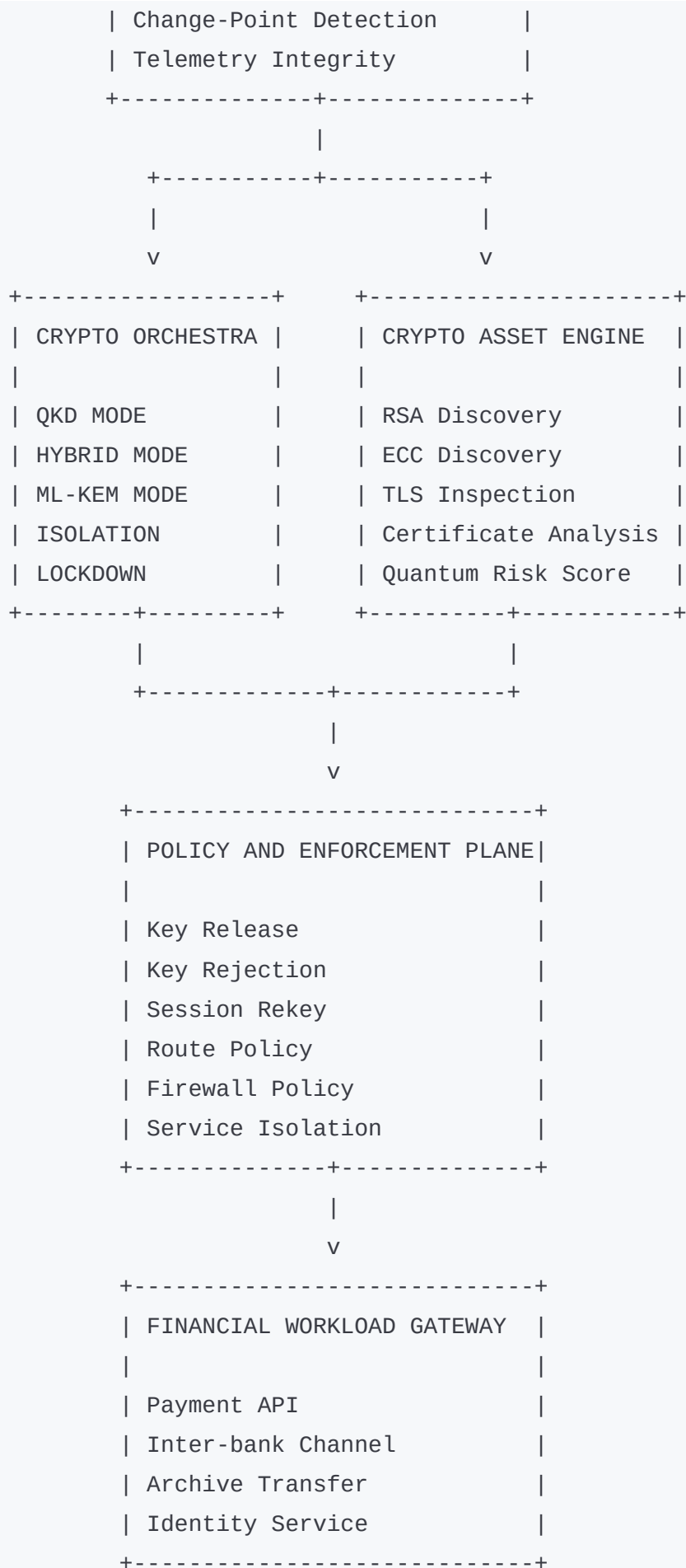
3. PRODUCT ARCHITECTURE



```

| Backend Identity |
| Job Verification |
| Measurement Acquisition |
| Provenance Metadata |
+-----+-----+
|
| v
+-----+-----+
| BB84 / QKD PROCESSING PLANE |
| |
| Basis Preparation |
| Measurement Processing |
| Sifting |
| Parameter Estimation |
| QBER |
| Reconciliation |
| Privacy Amplification |
| Key Acceptance |
+-----+-----+
|
| v
+-----+-----+
| QUANTUM TELEMETRY BUS |
| |
| QBER |
| QBER Trend |
| Error Distribution |
| Sifted Key Rate |
| Secret Key Rate |
| Key Pool |
| Backend Provenance |
+-----+-----+
|
| v
+-----+-----+
| CHANNEL TRUST ENGINE |
| |
| EWMA |
| CUSUM |
| Rolling Variance |
| Basis Error Analysis |

```



4. MODULE 1 - REAL QUANTUM EXECUTION LAYER

Objective

Remove Python pseudo-randomness from every component claimed as quantum-generated.

Requirements

Create a backend interface:

```
class QuantumBackend:
    def backend_name(self):
        pass

    def backend_type(self):
        pass

    def submit_circuit(self, circuit):
        pass

    def get_measurements(self, job_id):
        pass

    def get_provenance(self, job_id):
        pass
```

Implement:

```
IBMQuantumBackend
PhysicalQRNGBackend
QKDHardwareBackend
```

The IBM backend must submit circuits to an actual available IBM Quantum processor.

Every execution must retain:

```
backend_name
job_id
execution_timestamp
shot_count
circuit_hash
result_hash
```

The dashboard must display:

```
EXECUTION SOURCE: REAL QPU
BACKEND: <actual backend>
JOB ID: <actual job id>
CIRCUIT SHA-256: <hash>
RESULT SHA-256: <hash>
```

No hardcoded QPU name.

No fake job IDs.

No local fallback silently labelled as quantum hardware.

If the real QPU is unavailable:

```
QUANTUM BACKEND UNAVAILABLE
```

The system must not relabel local execution as real hardware.

Quantum entropy circuit

Prepare qubits using Hadamard gates and measure them.

```
|0> -- H -- M
|0> -- H -- M
|0> -- H -- M
...
```

Raw measurement output must pass through an extractor before security use.

Implement:

```
Raw QPU Bits
  |
Health Tests
  |
Toeplitz Extractor
  |
Extracted Entropy Pool
```

Add:

monobit health check
repetition count check
adaptive proportion check
entropy estimation
Toeplitz extraction

The QPU measurement source must not be marketed as a certified QRNG.

The correct claim is:

Quantum measurement-derived entropy acquired from a real quantum processor and processed through an extraction pipeline.

5. MODULE 2 - BB84 PROTOCOL ENGINE

Objective

Implement the complete BB84 protocol processing workflow.

Components

Alice Bit Generation
Alice Basis Selection
Quantum State Preparation
Bob Basis Selection
Measurement
Basis Announcement
Sifting
Parameter Estimation
QBER Calculation
Information Reconciliation
Privacy Amplification
Key Confirmation
Key Acceptance / Rejection

Key states

RAW
SIFTED
ESTIMATING

RECONCILING
AMPLIFYING
CONFIRMING
ACCEPTED
REJECTED
ZEROIZED

A key marked **REJECTED** must never be returned by the KMS.

A key marked **ZEROIZED** must not remain accessible through the application API.

Key identity

Every key receives:

key_id
session_id
channel_id
creation_time
source_backend
qber
reconciliation_leakage
privacy_amplification_ratio
final_key_length
state

Important scientific restriction

The system must separate:

PROTOCOL EXECUTION

from:

PHYSICAL QKD LINK

A real QPU circuit execution may validate quantum state preparation and measurement logic.

It must not be described as a fibre-based BB84 QKD deployment.

A physical QKD claim is permitted only when a QKD optical transmitter, receiver and quantum channel are connected.

6. MODULE 3 - INFORMATION RECONCILIATION

Objective

Remove the current paper limitation where reconciliation is missing.

Implementation

Implement Cascade-style interactive reconciliation.

Stages:

```
Sifted Key
  |
Random Permutation
  |
Block Partition
  |
Parity Comparison
  |
Binary Search
  |
Error Correction
  |
Additional Passes
  |
Corrected Key
```

Record:

```
passes
parity_messages
corrected_bits
leaked_parity_bits
reconciliation_efficiency
execution_time
```

The privacy amplification stage must account for reconciliation leakage.

Experiment

Compare:

No Reconciliation
vs
Cascade Reconciliation

Metrics:

Alice/Bob key agreement
remaining key length
reconciliation latency
information leakage
final secret length

7. MODULE 4 - PRIVACY AMPLIFICATION

Objective

Generate a reduced final key after QBER estimation and reconciliation.

Use a universal hash construction based on Toeplitz matrices.

Pipeline:

```
Corrected Key
  |
Estimate Leakage
  |
Determine Output Length
  |
Toeplitz Universal Hash
  |
Final Secret Key
```

Store the privacy amplification parameters with the key record.

Do not simply hash the complete sifted key and call it privacy amplification.

8. MODULE 5 - QUANTUM TELEMETRY PROTOCOL

Objective

Replace unauthenticated `/link_status` polling.

Protocol name:

QTP - Quantum Telemetry Protocol

Telemetry packet

```
{  
  "protocol": "QTP",  
  "version": 1,  
  "node_id": "alice-qkd-01",  
  "channel_id": "qch-01",  
  "session_id": "uuid",  
  "sequence": 10284,  
  "timestamp": 1783289212,  
  "qber": 0.061,  
  "qber_x": 0.058,  
  "qber_z": 0.064,  
  "sifted_key_rate": 24192,  
  "secret_key_rate": 11842,  
  "key_pool_bits": 8192,  
  "backend_type": "REAL_QPU",  
  "backend_job_id": "actual-job-id",  
  "nonce": "random-value",  
  "signature": "signature-value"  
}
```

Authentication

Use ML-DSA for node signatures.

Each QTP node has:

```
node_id  
ML-DSA public key  
certificate fingerprint  
allowed channel IDs  
policy role
```

Required protections

```
signature verification  
timestamp validation
```

```
sequence validation
nonce replay cache
node identity verification
channel authorization
message schema validation
rate limiting
```

Attack tests

Implement:

```
Replay old GREEN telemetry
Modify QBER
Modify key rate
Fake node identity
Replay nonce
Reverse sequence number
Delay RED telemetry
Flood telemetry endpoint
Malformed packet
Unauthorized channel ID
```

Every attack must generate an audit event.

9. MODULE 6 - TEMPORAL QUANTUM CHANNEL TRUST ENGINE

Objective

Move beyond:

```
QBER >= 11% = RED
```

The engine must analyze channel behaviour over time.

Input features

```
current QBER
X-basis QBER
Z-basis QBER
QBER EWMA
```

```
QBER slope
rolling variance
CUSUM positive drift
CUSUM negative drift
sifted key rate
secret key rate
key rate change
consecutive anomalies
telemetry freshness
```

Algorithms

Implement:

```
Static Threshold
EWMA
CUSUM
Rolling Variance
Change-Point Detection
Combined Evidence Engine
```

No neural network in version 1.

No LSTM.

No QML classifier.

The engine must remain explainable.

Channel states

```
TRUSTED
NOISY
DEGRADING
SUSPICIOUS
COMPROMISED
UNKNOWN
```

Decision output

```
{
  "state": "SUSPICIOUS",
```

```

"confidence": 0.87,
"evidence": [
  "QBER EWMA increased 3.1x above baseline",
  "positive CUSUM threshold exceeded",
  "secret key rate decreased 42%",
  "telemetry signature valid"
]
}

```

Every classification must contain evidence.

10. MODULE 7 - PHYSICAL CHANNEL TEST RIG

Objective

If the project is to claim real QKD, build an actual optical quantum communication test rig.

Required physical components

The team must obtain or access:

```

single-photon or attenuated optical source
optical encoding mechanism
quantum channel
basis selection optics
single-photon detection capability
timing and synchronization hardware
classical authenticated channel

```

Nodes

```

ALICE OPTICAL NODE
    |
    | quantum optical channel
    |
BOB OPTICAL NODE

ALICE <---- authenticated classical channel ----> BOB

```

Eve insertion point


```
Alice
|
Quantum Channel
|
[EVE TEST NODE]
|
Bob
```

Eve must physically alter or intercept the transmitted optical state for the experiment to be called a physical interception experiment.

Software-generated QBER injection is not sufficient for that claim.

Go/no-go rule

If the required optical hardware is unavailable before Quant-A-Thon:

Do not claim physical QKD.

Present the system as:

Real-QPU-backed quantum state execution with a complete BB84 protocol and security orchestration implementation.

The optical channel limitation must be explicitly disclosed.

11. MODULE 8 - REAL POST-QUANTUM CRYPTOGRAPHY

Objective

Remove the fake `os.urandom(32)` PQC placeholder.

Implement standardized ML-KEM.

Required algorithm:

ML-KEM-768

Functions:

```
keygen()
encapsulate()
```

```
decapsulate()
```

Store:

```
algorithm  
parameter_set  
public_key_fingerprint  
ciphertext_hash  
encapsulation_time  
decapsulation_time  
shared_secret_hash
```

Never log the shared secret.

Signature

Implement:

```
ML - DSA
```

Use ML-DSA for:

```
QTP node authentication  
policy signing  
migration plan signing  
audit checkpoint signing
```

12. MODULE 9 - HYBRID KEY COMBINER

Objective

Combine independent key sources.

Inputs:

```
QKD_ACCEPTED_KEY  
ML_KEM_SHARED_SECRET  
SESSION_CONTEXT
```

Derivation:

```
IKM =  
QKD_KEY  
||  
ML_KEM_SHARED_SECRET
```

Then use HKDF-SHA-256 with explicit domain separation.

Context:

```
SHORLYNOT-HYBRID-V1  
session_id  
channel_id  
alice_id  
bob_id
```

Output:

```
SESSION_ENCRYPTION_KEY
```

Modes:

```
QKD_ONLY  
ML_KEM_ONLY  
QKD_ML_KEM_HYBRID  
LOCKED
```

A rejected QKD key must never enter the hybrid combiner.

13. MODULE 10 - CRYPTO AGILITY ORCHESTRATOR

Objective

Automatically select the security mechanism according to evidence and policy.

Policy examples

```
TRUSTED QKD CHANNEL  
+  
VALID TELEMETRY  
+  
HEALTHY KEY POOL
```

→ QKD_ML_KEM_HYBRID

NOISY CHANNEL

+

ACCEPTABLE KEY SECURITY

→ HYBRID

→ INCREASE MONITORING

SUSPICIOUS CHANNEL

→ REJECT NEW QKD KEY

→ ML_KEM_ONLY

→ REKEY ACTIVE SESSION

COMPROMISED CHANNEL

→ ZEROIZE QKD MATERIAL

→ ML_KEM_ONLY

→ ISOLATE CHANNEL

→ SECURITY EVENT

INVALID TELEMETRY SIGNATURE

→ MARK NODE UNTRUSTED

→ REJECT TELEMETRY

→ ISOLATE TELEMETRY SOURCE

Requirement

The migration must happen in the actual communication client.

Changing a dashboard label is not migration.

The client must:

pause new application messages

complete key migration

derive new session key

```
confirm peer key state  
resume communication
```

Measure migration interruption time.

14. MODULE 11 - KEY HEALTH PLANE

Objective

Treat quantum keys as a finite security resource.

Monitor:

```
available key bits  
generation rate  
consumption rate  
rejected key rate  
key expiry  
reserved key volume  
estimated depletion time
```

Traffic priorities

```
P0 - emergency command  
P1 - payment settlement  
P2 - identity/authentication  
P3 - telemetry  
P4 - bulk archive
```

Policy example:

```
QKD key pool < 20%  
  
Reserve QKD material for P0 and P1.  
  
Move P2-P4 to ML-KEM.
```

Experiment

Measure:

critical traffic availability
key depletion time
QKD key utilization
PQC fallback frequency

Compare:

FIFO key allocation
vs
priority-aware key allocation

15. MODULE 12 - CRYPTOGRAPHIC ASSET DISCOVERY

Objective

Meet the YESIST12 financial migration requirement.

Build a real scanner.

Inputs

source-code directory
TLS endpoint
certificate directory
OpenSSL configuration
Nginx configuration
Apache configuration
Java keystore
PEM files

Detect

RSA
ECDSA
ECDH
DH
DSA
TLS versions
certificate signature algorithms

```
key sizes
hardcoded cryptographic algorithm names
```

Example result

```
{
  "asset_id": "PAYMENT-API-01",
  "location": "api.bank.local",
  "algorithm": "RSA",
  "key_size": 2048,
  "data_retention_years": 10,
  "criticality": "CRITICAL",
  "quantum_vulnerable": true
}
```

The scanner must analyze real test repositories and certificates.

No manually populated dashboard cards.

16. MODULE 13 - QUANTUM RISK SCORING

Objective

Rank financial assets for migration.

Inputs:

```
algorithm vulnerability
key size
asset criticality
data confidentiality lifetime
external exposure
transaction sensitivity
certificate expiry
migration complexity
```

Output:

```
IMMEDIATE
HIGH
```

MEDIUM

LOW

Example:

Customer KYC Archive

Algorithm: RSA-2048

Retention: 15 years

Exposure: Internet-facing transfer service

Criticality: Critical

HNDL Risk: IMMEDIATE

The scoring formula and weights must be published in the technical documentation.

Run sensitivity analysis on the weights.

17. MODULE 14 - FINANCIAL WORKLOAD GATEWAY

Objective

Demonstrate the architecture against actual financial-style application traffic.

Implement four services:

Payment API

Identity Service

Inter-Bank Message Service

Long-Term Archive Service

These are not fake UI pages.

Each service must expose a real network API.

Payment API

Send signed transaction objects.

```
{  
  "transaction_id": "TX-10292",  
  "sender": "BANK-A",  
  "receiver": "BANK-B",  
}
```



```
"amount": 125000,  
"currency": "INR",  
"timestamp": 1783289212  
}
```

Encrypt the transport payload using the current ShorlyNot session key.

Archive service

Transfer encrypted files.

Measure:

```
encryption throughput  
key migration interruption  
PQC handshake latency  
ciphertext overhead
```

Inter-bank service

Maintain a long-running encrypted channel.

During active traffic:

```
trigger channel degradation  
detect trust change  
reject QKD material  
perform cryptographic migration  
resume traffic
```

The judges must see the same TCP/application session survive a security-mode transition where technically feasible, or a controlled reconnect with measured interruption if the protocol requires it.

18. MODULE 15 - ACTUAL NETWORK ENFORCEMENT

Objective

Remove shell-variable failover.

Required actions:

```
real firewall rule modification
real process isolation
real route change
real interface policy
real session rekey
```

For Linux:

```
nftables
ip route
ip rule
network namespaces
```

For OpenWrt:

```
uci
ubus
nftables/firewall4
procd
```

The enforcement daemon must receive signed policy decisions.

Example:

```
CHANNEL_ISOLATE qch-01
```

The daemon verifies the policy signature before executing the action.

Record:

```
decision timestamp
verification timestamp
execution timestamp
completion timestamp
result
```

19. MODULE 16 - ADVERSARIAL SECURITY LAB

Objective

Attack the actual orchestration system.

Required attack categories:

Quantum-channel experiments

- high error condition
- progressive error increase
- burst error condition
- basis-dependent error pattern
- channel instability

If physical QKD hardware exists, these must originate from physical channel manipulation where possible.

Otherwise they must be clearly labelled channel-emulation experiments.

Telemetry attacks

- replay
- tampering
- spoofing
- delay
- reordering
- flooding

Cryptographic attacks

- rejected key reuse
- expired key request
- duplicate key ID
- invalid ML-KEM ciphertext
- invalid ML-DSA signature
- downgrade request

Orchestrator attacks

policy replay
fake migration command
state rollback
forced trusted state
lockdown recovery attempt

20. MODULE 17 - FORMAL SAFETY MODEL

Model the security state machine.

States:

INITIALIZING
TRUSTED
NOISY
SUSPICIOUS
MIGRATING
PQC_FALLBACK
ISOLATED
LOCKDOWN

Verify:

REJECTED QKD KEY IS NEVER RELEASED

ZEROIZED KEY IS NEVER REUSED

INVALID TELEMETRY CANNOT CREATE TRUSTED STATE

SEQUENCE NUMBER NEVER MOVES BACKWARD

LOCKDOWN CANNOT SELF-RECOVER

TRAFFIC DOES NOT RESUME BEFORE KEY CONFIRMATION

QKD KEY DOES NOT ENTER HYBRID COMBINER AFTER REJECTION

The implementation tests must map to these safety properties.

Formal verification is supporting evidence.

It must not replace implementation testing.

21. MODULE 18 - CRYPTOGRAPHIC MIGRATION PLANNER

Objective

Meet the YESIST12 migration-roadmap requirement.

Input:

```
Crypto Asset Inventory
+
Quantum Risk Scores
+
PQC Benchmark Results
+
Compatibility Information
```

Output:

```
PHASE 0 - DISCOVERY

PHASE 1 - IMMEDIATE HNDL PROTECTION

PHASE 2 - HYBRID PILOT

PHASE 3 - PQC MIGRATION

PHASE 4 - QKD PILOT FOR SELECTED LINKS

PHASE 5 - CRYPTO AGILITY OPERATIONS
```

Every recommendation must include:

```
asset
current algorithm
risk
recommended algorithm
migration priority
estimated performance impact
```

compatibility issue
rollback requirement

No LLM-generated migration advice is required.

Use deterministic rules and measured benchmark data.

22. REQUIRED EXPERIMENTS

Experiment A - Static QBER vs temporal detection

Compare:

11% Static Threshold
EWMA
CUSUM
Combined Trust Engine

Metrics:

detection rate
false positive rate
mean detection time
detection lead time

Experiment B - Noise versus progressive interception

Test multiple channel behaviours.

Measure:

classification accuracy
false Eve alarm rate
missed attack rate
detection latency

If no physical QKD link exists, label this experiment as channel emulation.

Experiment C - Key processing

Compare:

sifted key
reconciled key
privacy-amplified key

Measure:

agreement rate
leakage
final key length
processing latency

Experiment D - ML-KEM

Measure ML-KEM-768:

key generation
encapsulation
decapsulation
public key size
ciphertext size

Experiment E - Cryptographic migration

Test:

HYBRID → ML-KEM
QKD → HYBRID
QKD → ML-KEM
ANY → LOCKDOWN

Measure:

migration latency
application interruption
packet loss
failed messages

Experiment F - Telemetry security

Run all QTP attacks.

Measure:

```
attack rejection rate  
false rejection rate  
verification latency
```

Experiment G - Key resource allocation

Compare:

```
FIFO  
Priority-Aware Allocation
```

Measure:

```
critical service availability  
key pool lifetime  
QKD utilization
```

Experiment H - Financial workload

Run:

```
10,000 payment requests  
1 GB archive transfer  
1-hour inter-bank channel  
10,000 identity requests
```

Compare:

```
Classical  
ML-KEM  
Hybrid  
ShorlyNot Adaptive
```

Measure:


```
latency
throughput
CPU
RAM
bandwidth overhead
migration interruption
```

23. REQUIRED GRAPHS

The final project must automatically generate:

```
QBER over time
QBER EWMA trajectory
CUSUM trajectory
Detection lead-time CDF
False-positive curve
Noise vs interception confusion matrix
Secret key rate over time
Key pool depletion curve
Reconciliation efficiency
Privacy amplification reduction
ML-KEM latency distribution
Crypto migration latency
Application throughput
CPU utilization
RAM utilization
QTP verification latency
Quantum risk distribution
Migration priority matrix
```

No manually created graph values.

All figures must be generated from experiment CSV files.

24. REPOSITORY STRUCTURE

```
shorlynot/
|
├─ quantum/
```

```
|   ├── backends/
|   |   ├── ibm_qpu.py
|   |   ├── physical_qrng.py
|   |   └── qkd_hardware.py
|   ├── bb84/
|   |   ├── alice.py
|   |   ├── bob.py
|   |   ├── sifting.py
|   |   ├── qber.py
|   |   ├── cascade.py
|   |   ├── privacy_amplification.py
|   |   └── key_confirmation.py
|   └── provenance.py
|
|── telemetry/
|   ├── qtp.py
|   ├── signer.py
|   ├── verifier.py
|   ├── replay_cache.py
|   └── schemas/
|
|── trust/
|   ├── baseline.py
|   ├── ewma.py
|   ├── cusum.py
|   ├── change_point.py
|   └── evidence_engine.py
|
|── crypto/
|   ├── mlkem.py
|   ├── mldsa.py
|   ├── hybrid_kdf.py
|   ├── aesgcm.py
|   └── lifecycle.py
|
|── orchestra/
|   ├── policy.py
|   ├── state_machine.py
|   ├── migration.py
|   └── enforcement.py
|
```

```
├─ key_health/
|   ├─ pool.py
|   ├─ allocator.py
|   └─ predictor.py
|
├─ discovery/
|   ├─ source_scanner.py
|   ├─ certificate_scanner.py
|   ├─ tls_scanner.py
|   └─ config_scanner.py
|
├─ finance/
|   ├─ payment_api/
|   ├─ identity_service/
|   ├─ interbank/
|   └─ archive/
|
├─ enforcement/
|   ├─ linux_agent/
|   └─ openwrt_agent/
|
├─ attacks/
|   ├─ telemetry/
|   ├─ crypto/
|   ├─ orchestrator/
|   └─ channel/
|
├─ experiments/
|   ├─ runners/
|   ├─ datasets/
|   ├─ results/
|   └─ plots/
|
├─ formal/
|   └─ ShorlyNot.tla
|
├─ dashboard/
|
├─ tests/
|
└─ docs/
```

25. QUANT-A-THON BUILD PROFILE

For QT-3.5 enable:

```
REAL QPU BACKEND
BB84 ENGINE
QBER ANALYSIS
CASCADE
PRIVACY AMPLIFICATION
CHANNEL TRUST ENGINE
QTP
KEY REJECTION
ATTACK LAB
QUANTUM PROVENANCE
```

Pitch:

“ShorlyNot extends a BB84 implementation beyond static QBER display by treating quantum-channel measurements as authenticated temporal security telemetry. The system performs key processing, analyzes channel degradation, rejects compromised key material and provides reproducible provenance for quantum backend execution.”

Do not make finance the centre of this pitch.

Do not spend presentation time on crypto inventory.

26. YESIST12 BUILD PROFILE

Enable:

```
CRYPTO ASSET DISCOVERY
QUANTUM RISK ENGINE
ML - KEM
ML - DSA
HYBRID KEY COMBINER
CRYPTO AGILITY
SHORLYNOT QKD MODULE
KEY HEALTH PLANE
FINANCIAL WORKLOAD GATEWAY
MIGRATION PLANNER
NETWORK ENFORCEMENT
```

Pitch:

“ShorlyNot Q-Orchestra is a cryptographic agility control plane for financial infrastructure. It discovers quantum-vulnerable assets, measures migration risk, benchmarks post-quantum controls and autonomously transitions protected workloads between hybrid and quantum-assisted security modes according to verified security telemetry.”

27. NON-NEGOTIABLE CLAIM RULES

Never say:

“We implemented physical QKD”

unless photons were transmitted through a quantum communication channel between Alice and Bob.

Never say:

“Quantum random number generator”

for IBM QPU measurement output unless the implementation and validation support that exact claim.

Use:

“quantum measurement-derived entropy.”

Never say:

“AI detects Eve”

unless an AI model actually performs the classification.

Never say:

“ML-KEM”

unless a real ML-KEM implementation performs encapsulation and decapsulation.

Never say:

“network switched”

unless an interface, route or network path actually changed.

Never say:

“port migrated”

unless the server and client moved to the new port.

Never say:

“key destroyed”

if the application can still retrieve the key.

Never say:

“real quantum hardware”

without preserving backend execution provenance.

28. FINAL PRODUCT DEFINITION

ShorlyNot Q-Orchestra is not a BB84 dashboard.

It is not a banking UI with quantum branding.

It is not an AES application using a simulated QBER value.

It is a cryptographic security orchestration system composed of:

```
REAL QUANTUM EXECUTION PROVENANCE
+
COMPLETE BB84 KEY PROCESSING
+
AUTHENTICATED QUANTUM TELEMETRY
+
TEMPORAL CHANNEL TRUST ANALYSIS
+
REAL ML-KEM AND ML-DSA
+
HYBRID KEY COMPOSITION
+
AUTONOMOUS CRYPTOGRAPHIC MIGRATION
+
QUANTUM KEY RESOURCE MANAGEMENT
+
CRYPTOGRAPHIC ASSET DISCOVERY
+
FINANCIAL QUANTUM-RISK ASSESSMENT
+
REAL NETWORK ENFORCEMENT
```

The system's central contribution is:

“ShorlyNot transforms verified quantum-channel and cryptographic telemetry into enforceable runtime security decisions, enabling cryptographic migration, key-resource management and workload protection across hybrid quantum-safe infrastructure.”

That is the product to build.

...

One important thing, fam: **if you genuinely mean “100% real physical QKD,” software is no longer the blocker. Hardware is.** Real experimental BB84/QKD systems use optical sources, encoding/decoding optics, quantum channels and photon detection; real implementations are hardware-intensive even when engineered for resource efficiency. (arXiv)

For YESIST12, the PRD above strongly matches the supplied scope's demand for PQC evaluation, hybrid architecture, QKD feasibility, financial workload validation and a phased migration roadmap. [Quantum-Safe Finance.pdf](#) For QT-3.5, BB84 correctness, Eve detection, QBER interpretation and reproducibility remain the explicit scoring centre. [Problem Statement Quantathon'26.pdf](#)

My strongest recommendation is not to buy random laser parts and claim physical QKD. Build every classical/PQC/orchestration component for real, run the quantum measurement layer on an actual accessible QPU with verifiable job provenance, and make the physical-QKD backend a strict adapter unless you obtain proper optics and detectors. That version survives a technical judge opening your source code.

If you want, I can turn this PRD into a **30-day execution checklist mapped against your current** [iptable.zip](#) : **KEEP / DELETE / REWRITE / NEW FILE**, so you can start coding from the existing project instead of rebuilding blindly.

Sources

[Reconfigurable MDI-QKD and BB84 over 20 km optical channels via EOM-tailored weak coherent states](#)

arxiv.org

7/6/2026, 12:32:26 AM



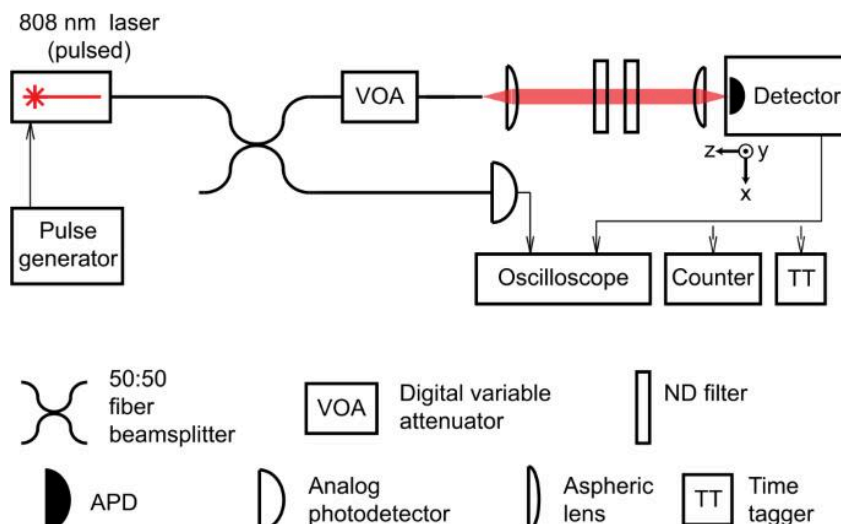
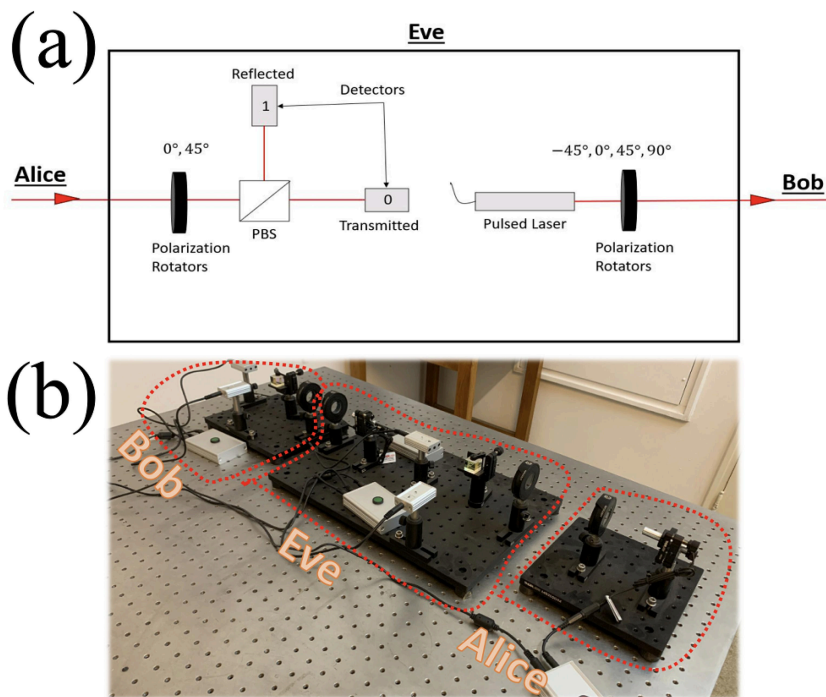
lets talk for now no need of code or stuff apart from my modem what are the hardware or what do i need?

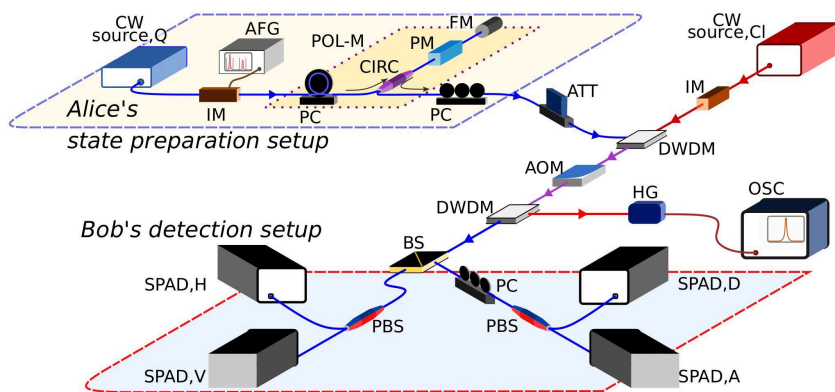
7/6/2026, 12:35:03 AM



Yeah, just talking hardware.

Apart from the D-Link modem, you have two paths. The answer depends on whether you want **real quantum hardware execution** or **actual physical BB84/QKD with photons**. Those are not the same thing.





For **your two hackathons**, I would build a hybrid physical setup with:

- **Your laptop** - RTX 4060/i7 machine is more than enough. This runs KMS, trust engine, QBER analysis, ML-KEM, experiments and orchestration.
- **D-Link DSL-2750U** - edge enforcement/router node.
- **2× Raspberry Pi 4/5 or mini PCs** - one acts as Alice, one as Bob. A Raspberry Pi 5 is convenient, but you do not need huge compute.
- **1× third Raspberry Pi / old laptop** - Eve/adversarial node. You can physically insert this between nodes for network and telemetry attacks.
- **USB-to-Ethernet adapters + Ethernet cables + small unmanaged switch** - lets us build a real isolated network and physically move traffic through Alice → router/enforcement → Bob.
- **A hardware entropy source** if we can access one. Alternatively, acquire quantum measurement-derived entropy from real QPU executions, with provenance. I would not call that a certified QRNG.

That setup already gives you **real networking, real ML-KEM, real AES-GCM, real key migration, real firewall enforcement, real telemetry attacks and real hardware nodes.**

But if you tell me:

"Akash wants photons actually travelling from Alice to Bob."

Then we're entering **physical QKD hardware**.

For a polarization BB84 setup, conceptually you need:

- pulsed laser source;

- optical attenuator;
- polarization controller;
- polarizing beam splitter;
- half-wave plates;
- quarter-wave plates if required by the encoding design;
- fibre couplers;
- optical fibre/channel;
- photon detectors;
- timing/synchronization hardware;
- Alice control electronics;
- Bob control electronics;
- authenticated classical communication channel.

Modern experimental BB84 implementations still fundamentally require an optical transmitter/channel/receiver and photodetection; recent work uses weak coherent states, modulators, optical filtering and time-resolved photon detection, while practical decoy-state work explicitly accounts for source and detector imperfections. ([arXiv](#))

The **big problem is the detector**.

A proper single-photon detector-typically an SPAD/APD-based detector or equivalent-is where this gets expensive and technically annoying.

Without photon-level detection, I do **not** want you saying:

"We built physical QKD."

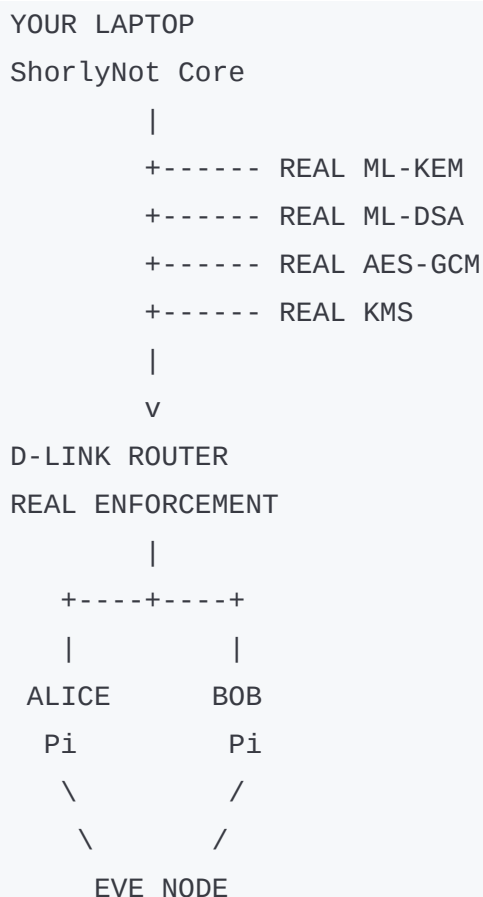
What I suggest for you

Build this physical topology:

```

REAL QUANTUM PROCESSOR
  |
Quantum Measurement Jobs
  |
  v

```



Then for Quant-A-Thon, **ask the organizers/mentors from [IBM Quantum](#), [National Quantum Mission](#), or [IITM C-DOT Samgnya Technologies Foundation](#) whether they can provide access to a QKD/quantum-optics lab or detector hardware.**

Because if they give you:

```

Laser/optical source
+
Attenuator
+
Polarization optics
+
Photon detector

```

we can try to connect a **physical QKD backend** to ShorlyNot.

My suggested **buy/collect list right now** is just: **2 Raspberry Pis, 1 old laptop/Pi for Eve, 3 USB-Ethernet adapters, Ethernet cables, a small switch, and your D-Link modem.** Don't randomly buy lasers and photodiodes yet 🙄. For real physical QKD, the exact optical architecture must be fixed first, because recent systems vary significantly between polarization and time-bin implementations. ([arXiv](#))

With your laptop + modem + 2 Pis + Eve node, we can already make 90% of ShorlyNot physically real. The remaining 10% is the actual quantum optical channel, and that's the expensive part.

Sources

[Reconfigurable MDI-QKD and BB84 over 20 km optical channels via EOM-tailored weak coherent states](#)

arxiv.org

7/6/2026, 12:35:12 AM



so its me and my friend so i have 2 laptop and my friend got one laptop and i got the dlink modem which we gonna use it somehow now addtioanlly what i need?

7/6/2026, 12:39:52 AM



Okay, with **3 laptops total + the D-Link DSL-2750U**, **do not buy Raspberry Pis**. You already have enough compute nodes.

I would physically assign them like this:

```
YOUR LAPTOP 1
ShorlyNot Core
KMS + Trust Engine + ML-KEM + QKD Processing

YOUR LAPTOP 2
ALICE NODE
Quantum sender / BB84 control

FRIEND'S LAPTOP
BOB NODE
Quantum receiver / secure financial service

D-LINK DSL-2750U
EDGE ENFORCEMENT
Firewall + isolation + route/network policy
```

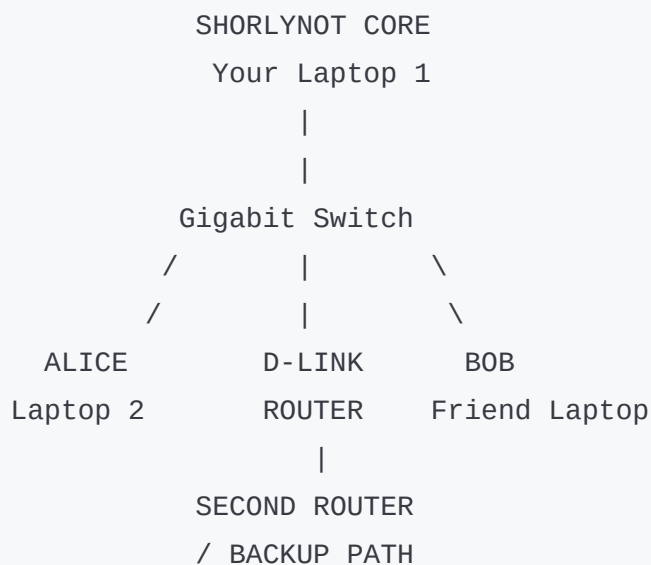
For Eve, **don't buy another machine**. Run Eve in an isolated VM or network namespace initially. For a physical man-in-the-middle networking test, one laptop can temporarily act as Eve with two network interfaces.

What I would get additionally

Hardware	Need	Why
3× USB-to-Ethernet adapters	YES	Multiple physical network paths
5-6× CAT6 cables	YES	Connect all nodes

Hardware	Need	Why
1× 8-port Gigabit unmanaged switch	YES	Physical test network
1× second cheap router/OpenWrt-supported router	HIGHLY recommended	Real path failover
2× USB flash drives	YES	Recovery/logging/firmware
1× USB TTL UART adapter - 3.3 V	YES	D-Link serial console/recovery
Dupont jumper wires	YES	UART connection
Multimeter	Recommended	Verify router UART pins/voltage
USB hub	Optional	Cleaner setup

So your lab becomes:



The **second router is actually important.**

Because right now your ShorlyNot claims:

PORT ROTATION
IP FAILOVER
NETWORK SWITCHING
LOCKDOWN

With one D-Link, network switching is difficult to prove properly.

With:

D-Link Router
+
Second Router

you can physically show:

```
PRIMARY PATH
Alice → D-Link → Bob

QKD CHANNEL TRUST DROPS
    ↓
ShorlyNot detects degradation
    ↓
QKD key rejected
    ↓
Crypto mode → ML-KEM
    ↓
Primary path isolated
    ↓
Traffic moves to Router 2
    ↓
Bob communication resumes
```

Actual packets. Actual route change. Actual firewall rules.

Not dashboard animation.

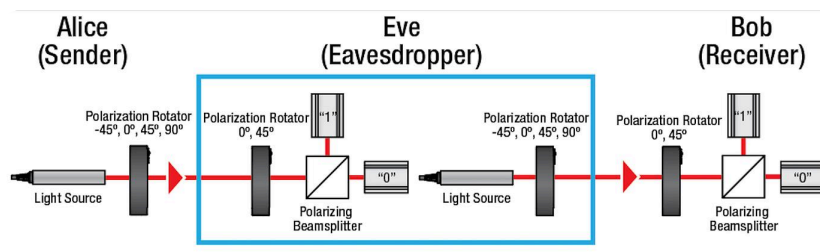
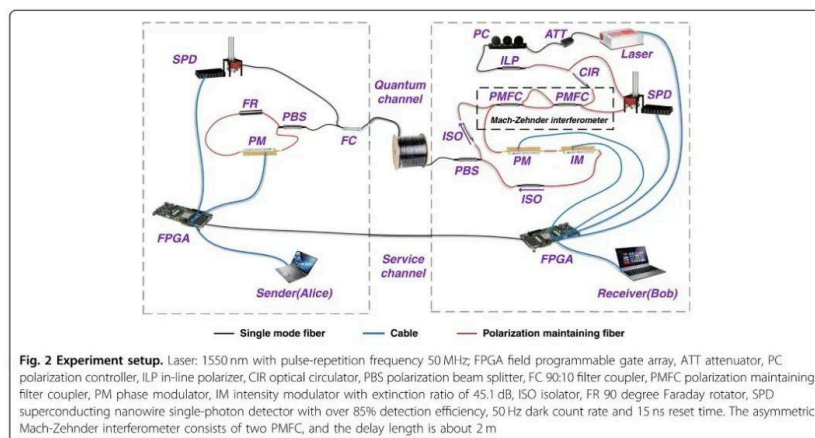
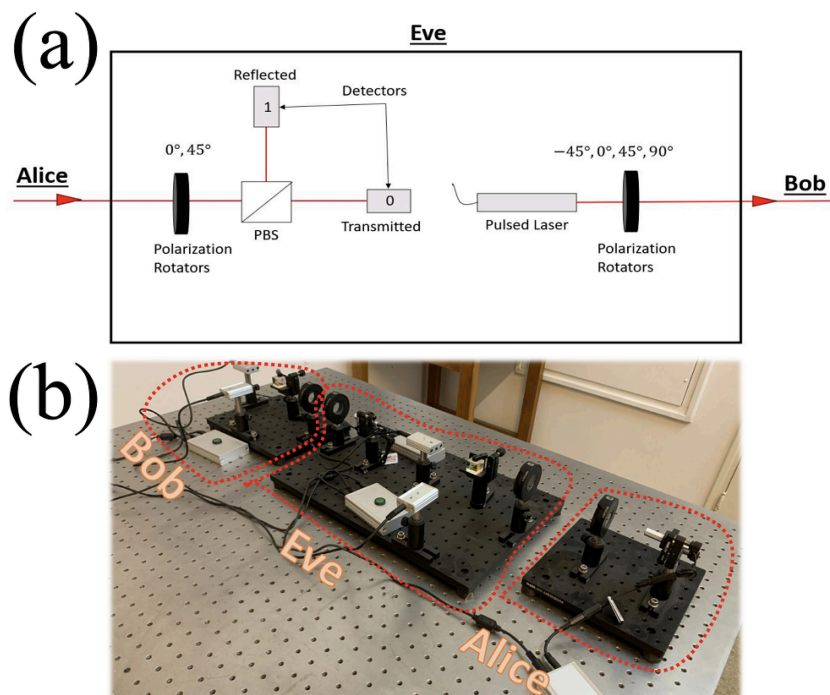
For the quantum part

This is where I don't want you buying random stuff yet.

If we want **physical BB84**, the next hardware category is roughly:

Optical source
Attenuation
Polarization encoding
Optical channel
Basis selection

Single-photon detection Timing/synchronization



The detector is the problem. A random Arduino laser + LDR setup is **not real QKD** 🤔

So right now my buy list for you is only: 3 USB-Ethernet adapters, an 8-port Gigabit switch, 5-6 CAT6 cables, one cheap second router, a 3.3 V USB-TTL UART adapter, jumper

wires, and ideally a multimeter.

You already have the expensive compute part.

Before buying the second router, send me the exact D-Link DSL-2750U hardware revision from the sticker. Then I can tell you what second router makes sense and design the **exact physical topology for both Quant-A-Thon and YESIST12.**

7/6/2026, 12:39:54 AM



ok i want it to be like the Dlink thing should play a crucial role here it shouldnt be here just for name sake

7/6/2026, 12:40:54 AM

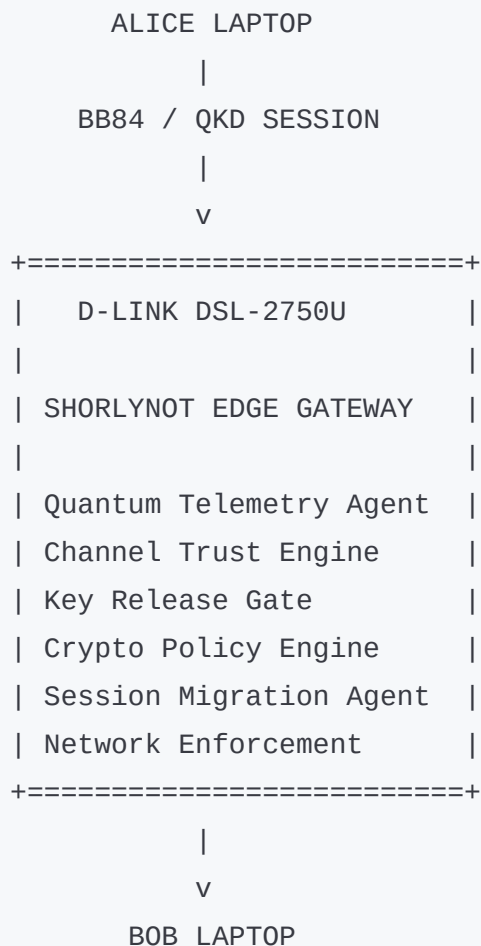


YES. Then I would redesign the whole architecture around the D-Link.

The D-Link should **not be “router that applies iptables rules.”**

Make it the **physical Quantum-Safe Enforcement Gateway.**

The D-Link becomes the heart of ShorlyNot



All secure traffic must physically pass through the D-Link.

If the D-Link is removed:

ShorlyNot's autonomous enforcement system stops working.

That's how crucial I want it.

What the D-Link actually does

The laptop does quantum processing and sends **signed quantum telemetry**:

```
QBER: 6.2%
QBER TREND: +1.8%
SKR: 420 bit/s
KEY_ID: QK-8291
CHANNEL: QCH-01
STATE: DEGRADING
SIGNATURE: VALID
```

The D-Link receives it.

But here's the important part:

The laptop does NOT decide the network response.

The **D-Link decides**.

```
Quantum Telemetry
  ↓
D-LINK
  ↓
Verify Telemetry
  ↓
Evaluate Trust
  ↓
Select Security Policy
  ↓
Enforce Network State
```

For example:

```
QBER NORMAL
↓
Allow QKD protected traffic

QBER DEGRADING
```

```
↓
Restrict non-critical traffic
Preserve quantum key pool

QBER SUSPICIOUS
↓
Block QKD key usage
Trigger ML-KEM migration

QBER COMPROMISED
↓
Terminate affected session
Zeroize edge key reference
Isolate Alice channel
Move Bob traffic to safe policy

TELEMETRY TAMPERED
↓
Block telemetry source
Enter PQC-only mode
```

Now the router is literally a **quantum-aware firewall**.

My favourite feature: Quantum-Aware Packet Firewall

Normal firewall:

```
SOURCE IP
DESTINATION IP
PORT
PROTOCOL
```

Your firewall:

```
SOURCE IP
DESTINATION IP
PORT
PROTOCOL
QKD CHANNEL TRUST
KEY STATE
```

```
CRYPTO MODE
TELEMETRY VALIDITY
```

Imagine:

```
ALLOW 10.0.0.2 → 10.0.0.3
IF:
    channel_trust >= 0.80
    key_state == ACCEPTED
    telemetry == VALID
    crypto_mode == HYBRID
```

Otherwise:

```
DROP
```

Call this:

Q-Aware Firewall Policy — QAFP

Your D-Link becomes a **security enforcement point where quantum telemetry affects real packet forwarding decisions.**

That is way more meaningful than `QBER RED → iptables DROP`.

Even better: Key-Gated Networking

This is something I really want you to build.

By default:

```
ALICE —X— BOB
```

```
NO ROUTE
```

Alice performs the BB84 process.

```
BB84
↓
Sifting
↓
QBER
```

↓
Reconciliation
↓
Privacy Amplification
↓
KEY ACCEPTED

The D-Link receives:

KEY_ID QK-9281
STATE ACCEPTED
CHANNEL TRUST 0.94

Only then:

D-Link
↓
Creates temporary firewall permission
↓
Alice ————— Bob

Call it:

Quantum Key-Gated Network Access

The firewall lease could be:

SESSION: S-8291
KEY: QK-9281
ALICE: 192.168.1.10
BOB: 192.168.1.20
VALID: 120 seconds

After 120 seconds:

LEASE EXPIRED
↓
FIREWALL CLOSED

New accepted key required.

So visually in the hackathon:

```
NO VALID QUANTUM KEY
```

```
Alice → Bob
```

```
BLOCKED BY D-LINK
```

Run BB84.

```
QBER: 2.8%
```

```
KEY ACCEPTED
```

Immediately:

```
D-LINK POLICY INSTALLED
```

```
Alice → Bob
```

```
CONNECTED
```

Then activate Eve.

```
QBER: 18.4%
```

```
KEY REJECTED
```

D-Link:

```
SESSION REVOKED
```

```
POLICY REMOVED
```

```
CONNECTION TERMINATED
```

```
SOURCE ISOLATED
```

Actual packets stop.

Bro, this is your demo.

Not graphs.

Not dashboard.

The judge can literally have a terminal running:

```
ping Bob
```

Before key:

```
Request timed out
```

Accepted BB84 key:

```
Reply from Bob  
Reply from Bob
```

Eve attack:

```
Request timed out
```

Then ShorlyNot migrates:

```
ML-KEM-768 ESTABLISHED
```

D-Link:

```
PQC FALLBACK POLICY ACTIVE
```

And:

```
Reply from Bob
```

That's a **killer live demonstration**.

The D-Link also owns the crypto state

I would put a tiny state table on it:

```
SESSION TABLE
```

```
S001
```

```
Alice → Bob
```

```
Channel: QCH-01
```

```
QBER: 2.7%
```

```
Key: QK-891
```



```
Mode: QKD+ML-KEM
Trust: 0.94
Policy: ALLOW
```

```
S002
Alice → Finance
Channel: QCH-02
QBER: 14.2%
Key: REJECTED
Mode: ML-KEM
Trust: 0.21
Policy: PQC_ONLY
```

The router **never stores raw QKD keys if its hardware is too constrained.**

It stores:

```
KEY ID
KEY STATE
CHANNEL
TRUST
CRYPTO MODE
EXPIRY
```

That's safer and realistic.

Then your architecture becomes genuinely clean

```
LAPTOP 1
QUANTUM PLANE

BB84
Quantum execution
QBER
Reconciliation
Privacy amplification
    |
    | SIGNED QTP
    v
D-LINK
ENFORCEMENT PLANE
```

Verify telemetry
Trust evaluation
Key gating
Quantum firewall
Crypto policy
Session control
Network isolation
|
v

LAPTOP 2 / 3
APPLICATION PLANE

Alice
Bob
Financial services

Your novelty angle becomes

ShorlyNot introduces an edge enforcement architecture in which authenticated quantum-channel telemetry and quantum-key lifecycle state are translated into real-time packet forwarding and cryptographic access policies on resource-constrained legacy network hardware.

Notice what I'm **not** claiming:

invented QKD

or:

invented quantum firewall

The research question is:

Can legacy network infrastructure be transformed into a quantum-aware enforcement point that gates network access using quantum-channel trust and key lifecycle state?

That's actually the direction I'd push hard.

And Akash, with this design, **do not buy a second router yet.**

Your three laptops + D-Link are enough.

The next thing I need is the **exact DSL-2750U H/W revision**. Because if we can get firmware-level control on your revision, I would make **QAFP + Quantum Key-Gated Network Access the centrepiece of ShorlyNot**. The D-Link stops being a prop and becomes the physical enforcement plane of the whole project.

7/6/2026, 12:40:56 AM



my doubt is can i use openwrt to make this as the firmware?

7/6/2026, 12:42:43 AM



Thinking

- **Remembering**
Checking router hardware revision
 - **Remembering**
Confirming firmware compatibility
 - **Remembering**
Determining custom gateway constraints
 - **Remembering**
Reviewing router model specifications
 - **Remembering**
Reviewing router model specifications
- ✓ Thought for 11s

Yes. That is exactly the right idea.

OpenWrt is a Linux-based router OS designed to replace vendor firmware and supports custom packages, daemons, firewall configuration, networking logic, and image builds. Resource-constrained OpenWrt devices have also been used for automated network-management services in published systems work. ([arXiv](#))

But there is one important distinction:

OpenWrt



BASE OPERATING SYSTEM

ShorlyNot EdgeOS



YOUR CUSTOM FIRMWARE IMAGE
BUILT USING OPENWRT

So I **wouldn't pitch it as:**

"We installed OpenWrt on a D-Link."

I would pitch it as:

"We developed ShortlyNot EdgeOS, a custom OpenWrt-derived firmware image that transforms a legacy D-Link gateway into a quantum-telemetry-aware security enforcement node."

The D-Link's role becomes:

D-LINK DSL-2750U
SHORLYNOT EDGEOS
OpenWrt-derived firmware
qtpd
Telemetry verification
qtrust
Channel trust state
qgate
Key-gated network access
qorchestra
Crypto policy enforcement
firewall4 / nftables
Real packet enforcement

And **yes, these can be actual firmware components** rather than laptop scripts.

For example, `qtpd` receives authenticated quantum telemetry from your QKD node. `qtrust` maintains only the channel/key state that the constrained router needs. `qgate` opens or revokes network leases based on accepted key state. The enforcement layer modifies real firewall and forwarding policy.

So:

NO ACCEPTED KEY

Alice —X— Bob

KEY ACCEPTED

Alice ——— Bob

QBER / TRUST COMPROMISED

Alice —X— Bob

ML-KEM FALLBACK ESTABLISHED

Alice ——— Bob

The **D-Link** physically controls whether packets pass.

That is not a dashboard simulation.

But your DSL-2750U is the deciding factor

From our earlier project context, your unit was noted as using a **Broadcom BCM63281**. The DSL-2750U has multiple hardware revisions, and OpenWrt support is **revision-specific**. I do **not** want to tell you “flash this image” based only on the model name.

Also, if your exact target is one of the old **4 MB flash / 32 MB RAM class** units, we need to build EdgeOS like embedded firmware, not like a normal Linux server:

NO Python

NO Flask

NO Qiskit

NO ML models

NO local database

NO giant LuCI dashboard

Instead:

C daemon / tiny shell components
ubus
procd
UCI
firewall integration
small signed binary messages
in-memory state table

That constraint is actually **good for your research angle**:

Can authenticated quantum security telemetry be converted into packet-level enforcement decisions directly on resource-constrained legacy networking hardware?

So yes bro: **OpenWrt can be the firmware foundation, and I think ShorlyNot EdgeOS should be the actual custom firmware we build.**

The only thing I need before saying “**your exact modem can run it**” is the exact **H/W Ver. from the DSL-2750U sticker**. Send me just that text or a photo. Then I'll verify the exact target and tell you **flashable OpenWrt base vs custom port required vs no-go**.

Sources

[A Resource Efficient Implementation of the RESTCONF Protocol for OpenWrt Systems](#) arxiv.org

7/6/2026, 12:42:45 AM



qnlabs does something similar
<https://www.qnulabs.com/quantum-key-distribution> with
armos

7/6/2026, 12:46:48 AM



Yes - and this is important. You spotted a real overlap.

I checked QNu Labs' official Armos QKD page properly.

Armos already does part of what our earlier ShorlyNot framing was trying to claim. It is a physical QKD appliance that sends quantum states over fibre, generates shared symmetric key material, monitors QBER, discards affected key bits when QBER becomes unacceptable, halts key generation, alerts operators, and resumes generation when conditions recover. It also integrates QKD as an overlay with existing cryptographic infrastructure and exposes standard key-management interfaces. (QNulabs)

So this:

```
QBER HIGH
↓
DETECT EVE
↓
REJECT KEY
↓
ALERT
```

is absolutely not our innovation.

And even this:

```
QKD
↓
KMS
↓
AES
↓
Enterprise network
```


is not enough. QKD key-management literature has already studied layered, policy-driven enterprise integration and backend-independent key services. ([arXiv](#))

But I think this actually helps us

Look carefully at how Armos is positioned.

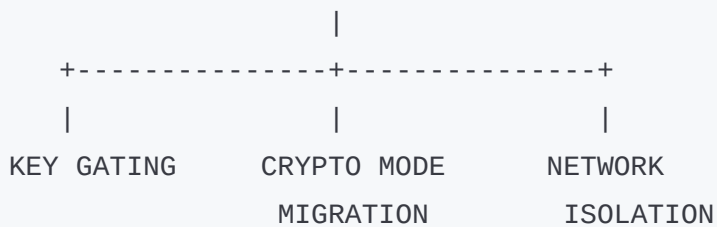
```
ARMOS  
  
PHYSICAL QKD APPLIANCE  
  ↓  
GENERATE QUANTUM KEYS  
  ↓  
QBER BAD?  
  ↓  
STOP KEY GENERATION  
DISCARD AFFECTED BITS  
ALERT  
  ↓  
QBER NORMAL?  
  ↓  
RESUME
```

That behaviour is described directly by QNu. ([QNulabs](#))

ShorlyNot should not compete with Armos.

We should sit **above and at the edge of systems like Armos.**

```
      QKD VENDOR  
Armos / Other QKD System  
  |  
  QKD / KMS telemetry  
  |  
  v  
SHORLYNOT CONTROL PLANE  
  |  
"What should the network do?"  
  |  
  v  
SHORLYNOT EDGEOS  
D-LINK ROUTER
```



That's the distinction

Armos asks:

Is the quantum channel safe enough to generate keys?

ShorlyNot asks:

Given the quantum security state, what should the existing network infrastructure autonomously do now?

That's a much better project.

For example, Armos detects abnormal QBER and stops risky key generation. ([QNulabs](#))

ShorlyNot receives:

```
CHANNEL qch-01
KEY GENERATION: HALTED
QBER: 14.7%
KEY POOL: 12%
```

Then the **D-Link itself** decides:

```
Payment traffic
→ ML-KEM hybrid migration

Admin traffic
→ isolate

Archive transfer
→ pause

Critical command traffic
```

→ consume reserved accepted QKD keys

New sessions

→ PQC only

Suspected QKD node

→ network quarantine

Armos doesn't appear to position the router as a quantum-state-aware packet-policy enforcement node on its public product page. Its public description emphasizes QKD key generation/distribution, QKDN lifecycle management, integration APIs, alerts, and SOC/SIEM visibility. ([QNulabs](#))

I would be careful and phrase our gap as:

“We investigate edge enforcement driven by authenticated QKD operational telemetry.”

Not:

“Nobody has ever done quantum-aware network enforcement.”

Because SDN/NFV QKD orchestration research already exists, including secure NFV orchestration over SDN-controlled optical networks and more recent standards-aware quantum security controllers. ([arXiv](#))

This changes my view on the D-Link

I think the **legacy D-Link is now even more important.**

Your story becomes:

QKD vendors can deploy specialised quantum appliances. But an enterprise still owns thousands of existing routers, gateways and branch-network devices. Can legacy edge infrastructure become an active enforcement participant in a quantum-safe network?

Then:

QKD SYSTEM

|

| standardized security telemetry

```

      v
DSL-2750U
2010-era legacy router
      |
SHORLYNOT EDGEOS
      |
Quantum-aware enforcement

```

That is actually a **beautiful SDG 9 / YESIST12 story**.

You aren't saying:

"We made a cheaper Armos."

That would get destroyed 😭

You're saying:

"We built the enforcement layer that lets legacy network infrastructure react to QKD and post-quantum security state."

And this is where I think we need to alter the PRD again.

Don't make our custom QTP protocol completely proprietary. QNu publicly says Armos integrates through standard interfaces including ETSI QKD interfaces, KMIP, and PKCS#11-style key-management integration. ([QNulabs](#))

We should build:

```

SHORLYNOT ADAPTER LAYER

ETSI QKD Adapter
KMIP Adapter
PKCS#11 Adapter
Native ShorlyNot Telemetry Adapter
      ↓
NORMALIZED QUANTUM SECURITY STATE
      ↓
EDGEOS

```

Now during the pitch you can say:

"ShorlyNot is vendor-agnostic. Our current BB84 backend is one telemetry source. The enforcement architecture is designed so commercial QKD/KMS systems can be integrated through standards-oriented adapters."

That is **way more industry-ready for the IEEE YESIST12 2026 [Capgemini](#) problem.**

My current view

Armos = quantum key infrastructure.

ShorlyNot = quantum-aware edge enforcement and crypto-agility infrastructure.

And fam, I actually think this is **stronger than the project we were discussing 20 minutes ago** because we now have a very clear answer when a judge says:

"QNu Labs already has Armos. Why do I need you?"

Your answer is:

"Armos protects the generation and distribution of quantum keys. ShorlyNot operationalizes the resulting security state at the legacy network edge. We turn QKD health, key availability, and cryptographic trust into enforceable packet, workload, and migration policy on infrastructure enterprises already own."

That's the lane. Do not clone Armos.

I think we should now do a **direct competitive gap map: Armos vs Toshiba QKD vs ID Quantique vs Cisco/ETSI QKD integration research vs ShorlyNot**, and kill every feature in our PRD that is already commodity or academically saturated. That will show us the **3-5 features that are actually worth building for both hackathons.**

Sources

[Quantum Key Distribution\(QKD\) Solution- Armos | QNu Labs](#) qnulabs.com

[Quantum Key Distribution Networks -- Key Management: A Survey](#) arxiv.org

7/6/2026, 12:46:51 AM



the ip table is real innovative thing right no?

7/6/2026, 12:47:33 AM



Yes - but with one correction.

`iptables` **itself is absolutely NOT innovative.** It is just Linux firewall tooling.

Your potentially innovative part is:

using authenticated QKD/QBER + key-lifecycle state as external security context to dynamically create and revoke packet-forwarding policy on a resource-constrained legacy router.

That distinction is everything.

Existing research already has **QKD-aware SDN controllers**, real-time quantum-parameter monitoring, rerouting, DDoS mitigation, and even next-generation firewalls consuming QKD keys through ETSI APIs. For example, a 2018 experimental system used real-time quantum parameters with SDN for QKD-network DDoS mitigation; dynamic QKD-aware SDN rerouting has also been demonstrated; and a 2024 100 Gbps QKD-secured IPsec deployment fed QKD keys into next-generation firewalls through ETSI-QKD-014. ([arXiv](#))

So **don't claim:**

"First quantum firewall."

or:

"First firewall controlled by QBER."

That could get attacked.

Your stronger angle is this:

NORMAL FIREWALL

IP

```
PORT
PROTOCOL
CONNECTION STATE
    ↓
ALLOW / DROP
```

versus:

```
SHORLYNOT  EDGEOS

IP
PORT
PROTOCOL
    +
QKD CHANNEL STATE
KEY ACCEPTANCE STATE
KEY EXPIRY
TELEMETRY INTEGRITY
CRYPTO MODE
    ↓
TEMPORARY NETWORK LEASE
    ↓
ALLOW / REVOKE / ISOLATE
```

The key-gated networking part is what I would push.

Imagine the D-Link starts with:

```
Alice → Bob = BLOCKED
```

Then ShorlyNot reports:

```
KEY_ID: QK-291
STATE: ACCEPTED
QBER: 2.7%
TELEMETRY: VALID
EXPIRY: 120 SEC
```

EdgeOS creates a **temporary packet-forwarding lease bound to that key state:**

SESSION S01
Alice → Bob
KEY QK-291
ALLOW
TTL 120s

If the key expires:

QK-291 → EXPIRED
D-Link → REVOKE FORWARDING

If QKD rejects the key:

QK-292 → REJECTED
D-Link → NEVER OPEN PATH

If an accepted key becomes operationally invalid because the channel/session is revoked:

SESSION REVOKED
D-Link → REMOVE POLICY
D-Link → TERMINATE FLOW

Then real packets stop.

That is much stronger than your current `QBER RED → iptables DROP`.

I'd name the actual mechanism:

Quantum Key-Bound Network Lease - QKBNL

The claim becomes:

“ShorlyNot binds network forwarding authorization to the lifecycle state of quantum-derived key sessions, enabling constrained edge gateways to issue and revoke temporary packet-forwarding leases from authenticated QKD security telemetry.”

Now I actually like this.

Because **Armos/QKD systems protect key generation**. QNu Labs' [Armos QKD](#) focuses on quantum key infrastructure.

Existing SDN work largely focuses on **QKD-network resource allocation, routing, orchestration, or quantum-secured tunnels.** ([arXiv](#))

Your research question can be narrower:

Can quantum-key lifecycle state act as a network authorization primitive on legacy edge infrastructure?

That is the innovation I would bet ShorlyNot on.

Not `iptables` .

Not BB84.

Not QBER detection.

The binding: `KEY STATE → NETWORK ACCESS LEASE` .

And your old-ass D-Link actually makes the story better 🤖 because you're proving this enforcement model on constrained legacy infrastructure rather than a cloud SDN controller.

I would make **QKBNL the centrepiece of the entire project and paper**, then put adaptive QBER/noise-vs-Eve detection as the intelligence feeding it.

Sources

[Experimental Demonstration of DDoS Mitigation over a Quantum Key Distribution \(QKD\) Network Using Software Defined Networking \(SDN\).](#)

arxiv.org

7/6/2026, 12:47:36 AM



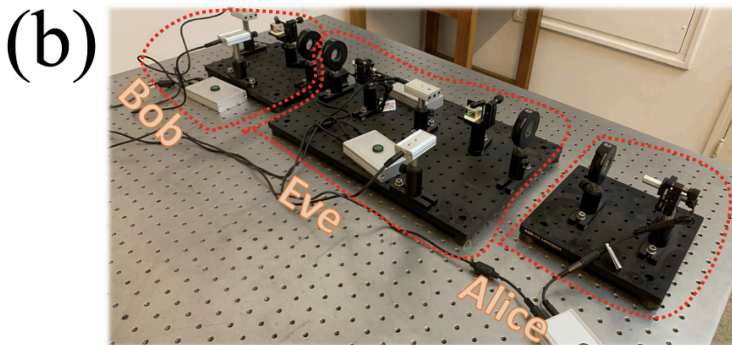
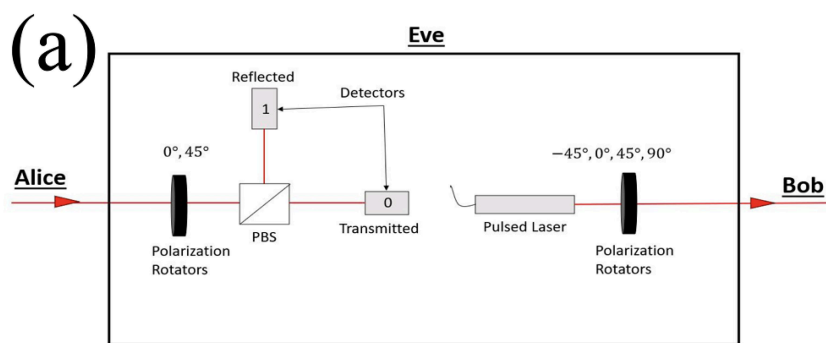
ok coming back to qunatum hardware part apart from modem what should i do?

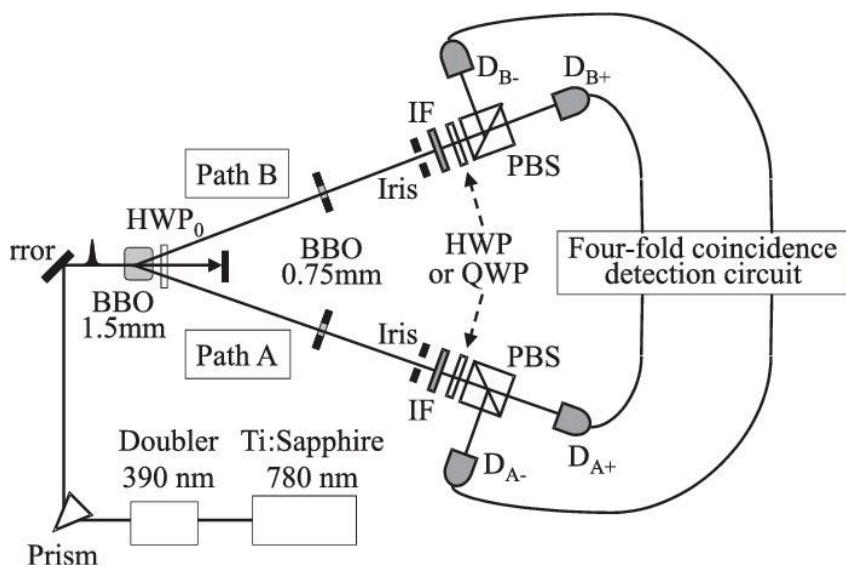
7/6/2026, 12:48:41 AM



Yeah. **Coming back purely to the quantum hardware part:** with your **3 laptops + D-Link**, I would **not buy more normal computers or routers now**.

The real missing piece is a **quantum optical channel**. Physical BB84/QKD requires special-purpose optical equipment; a cloud QPU is real quantum hardware, but it does not turn your laptops into a physical Alice-to-Bob QKD link. ([Wikipedia](#))





What I suggest you physically build

ALICE LAPTOP

|

Alice Control Electronics

|

OPTICAL TRANSMITTER

|

| PHOTON / OPTICAL CHANNEL

|
EVE INSERTION POINT
|
OPTICAL RECEIVER
|
Bob Control Electronics
|
BOB LAPTOP
|
D-LINK SHORLYNOT EDGEOS

For that, your **quantum hardware shopping/access list** is roughly:

Hardware	Purpose
Pulsed laser diode/source	Alice optical source
Optical attenuator	Reduce pulse intensity
Polarization encoder/modulator	Encode BB84 states
Half-wave plates	Polarization control
Polarizing beam splitter	Bob basis/measurement optics
Fibre couplers	Optical connections
Optical fibre spool	Quantum channel
Single-photon detector / SPAD	Detect photon events
Timing/synchronization hardware	Match transmission and detection
Microcontroller/FPGA	Control Alice/Bob optics

The **single-photon detector is the main blocker**. Real experimental QKD systems use photon-level optical sources/detection and actual quantum channels; even compact research implementations remain specialised hardware systems. ([arXiv](#))

What I would do in your situation

Do not buy this entire optical stack yourself immediately.

Your hackathon is associated with National Quantum Mission and quantum-technology partners. India already has active QKD infrastructure and lab work, including C-DOT quantum communication efforts. ([Wikipedia](#))

I would contact your Quant-A-Thon organisers and say:

"We are developing a quantum-telemetry-driven edge enforcement gateway for BB84/QKD networks. Our software, cryptographic orchestration and OpenWrt enforcement plane run on physical nodes. We are looking for temporary access to a QKD or quantum communication testbed so we can integrate real QBER and key-lifecycle telemetry instead of a simulated optical channel."

If they give you **QKD lab access**, that's the jackpot.

Then:

```
REAL QKD HARDWARE
    ↓
REAL QBER
REAL KEY STATE
    ↓
SHORLYNOT
    ↓
D-LINK EDGEOS
    ↓
REAL PACKET ENFORCEMENT
```

That's a technically honest end-to-end system.

If they don't give you lab access

Then my second choice is **partner/API integration with a real QKD system**, such as a university testbed or a vendor-oriented interface. [QNu Labs](#) already operates in the Indian quantum-security ecosystem, while commercial QKD systems exist specifically because physical QKD needs specialised equipment. ([Wikipedia](#))

Use the real QKD hardware's telemetry/key interface as input to your D-Link.

My recommendation for you right now

Buy nothing quantum yet.

First, try to obtain **QKD lab/testbed access** through RIT, IITM/C-DOT Samgnya, NQM contacts, or the hackathon mentors. Your D-Link is the innovation test target; you don't need to reinvent a commercial optical QKD appliance like Armos.

If they say **“we can give you optical lab access”**, we design the physical BB84 rig around the equipment they already have.

If they say **no**, then we can explore a low-cost optical prototype-but I will clearly separate **physical optical BB84 demonstration** from **security-grade QKD**.

For your project, I strongly prefer `real QKD testbed → ShorlyNot → D-Link EdgeOS` **over** **buying ₹₹₹ worth of random quantum optics.**

Sources

[Quantum key distribution](https://en.wikipedia.org) en.wikipedia.org

[Quantum Key Distribution Using a Quantum Emitter in Hexagonal Boron Nitride](https://arxiv.org) arxiv.org

7/6/2026, 12:48:48 AM